

A Lean 4 Certificate of Scott’s Lectures on a Mathematical Theory of Computation (1981) generated by LLM and registered by Palomar

Lars Ericson¹

¹ORCID: 0000-0001-8299-9361

¹Catskills Research Company

¹lars.ericson@catskillsresearch.com

September 12, 2026

Github: <https://github.com/catskillsresearch/scott1980>

Palomar Registration:

<https://palomar-registry.org/entry?id=PALOMAR-2026-08-24-000001&version=2>

Abstract

In November 1969, Dana Scott formulated a mathematical program to construct a non-degenerate, purely mathematical model (D_∞) for Alonzo Church’s untyped λ -calculus — the construction later published in *Continuous Lattices* (1972). That paper is often described as giving the first such model; this formalization does not adjudicate historical priority. Scott’s 1972 framework relied on point-set topology, which remained a barrier for computer scientists seeking a practical tool for programming-language semantics.

When Scott delivered his Oxford lectures in Michaelmas Term 1980 — published May 1981 as *Lectures on a Mathematical Theory of Computation* (Technical Monograph PRG-19) — he pivoted to discrete information presented as *domains*. That presentation was intended to be more accessible without training in point-set topology and lattice theory.

This Lean 4 library formalizes the numbered definitions, theorems, examples, and exercises of PRG-19, with one documented deferral: Exercise 8.17 Part 2 (whether $\mathcal{U} + \mathcal{U}$, $\mathcal{U} \times \mathcal{U}$, or $\mathcal{U} \rightarrow \mathcal{U}$ is isomorphic to $\mathcal{U}$), which Scott himself left unresolved for the function-space case. Data constructions are kept choice-free where possible. Prop-level results may use `Classical.choice`; axiom audits record the footprint. Palomar Comparator selects the principal sourced theorem `theorem_8_8` and two new formal corollaries, `P4_embeds` and `Ssys_embeds`; their permitted footprint is `{propext, Classical.choice, Quot.sound}`.

Proofs in this note are summarized at the level of their main constructions. Short representative Lean fragments are copied from the library. The Appendix gives a short description of each library file and a link to that file in the Palomar-preserved copy of this repository.

0.0.1 Palomar compared claim (first sentence of Theorem 8.8)

The principal sourced result is **only the first sentence of Theorem 8.8**: every countable neighbourhood system $\mathcal{D}$ embeds as a subdomain of $\mathcal{U}$ ($\mathcal{D} \trianglelefteq \mathcal{U}$, Lean `theorem_8_8`). Comparator additionally selects two new formal corollaries, not separately stated by Scott: specializing the theorem to Example 1.5 gives `P4_embeds`, and specializing it to Exercise 7.22’s system S gives `Ssys_embeds`. Scott’s effective projection-pair sentence and finitary-projection correspondence are proved in the library as `theorem_8_8_b` / `theorem_8_8_c` but are **not** Comparator targets. Scott assumes from the outset that the master token set Δ is non-empty; `NeighborhoodSystem.master_nonempty` now enforces that assumption structurally. See `Challenge.lean` and `formalization.yaml`.

1 Introduction

To make domain theory accessible, the 1980 monograph introduces three key conceptual and structural shifts:

1.1 The Information-Theoretic Ordering

In contrast to the topological open sets of 1972, the 1980 lectures treat domains strictly as partially ordered sets (posets) representing states of incomplete information. An element within a domain is framed as a “partial description” of a computation. The ordering relation ($\sqsubseteq$) is explicitly interpreted as approximation: $x \sqsubseteq y$ means x contains less information than, or approximates, y .

1.2 Neighbourhood Systems and Finite Approximations

To bypass the complexities of continuous geometric spaces, Scott introduced **Neighbourhood Systems**. He recognized that real-world computing machines only ever interact with finite, checkable tokens of data. In this framework, an infinite computational process (such as an infinite stream or a complex recursive function) is defined as the limit of an ever-tightening sequence of these finite neighbourhoods. This shifted the underlying mathematics away from general topology and toward formal logic and order theory.

1.3 Solving Universal Recursive Domain Equations

While Scott’s 1969 discovery was a specialized solution to the specific self-referential equation $D \cong [D \rightarrow D]$, the 1980 monograph provides a universal factory blueprint. Scott uses inverse limits over Directed-Complete Partial Orders (CPOs) to solve arbitrary recursive domain equations. This generalized framework allowed computer scientists to give rigorous mathematical meaning to standard recursive computer data structures, such as lists, trees, and stream types.

1.4 Formalization Target: Consolidating “Scott Domains”

This Lean 4 artifact formalizes the mathematical objects that these 1980 lectures ultimately standardized for the computer science community, known today as **Scott Domains**. A Scott Domain is characterized as a poset that is: 1. **Directed-Complete (CPO)**: Every directed subset has a least upper bound, ensuring that infinite computations have well-defined limits. 2. **ω -algebraic**: Every element in the domain can be represented as the supremum of a countable set of compact (finite) elements, mirroring how infinite data is built from finite tokens. 3. **Consistently Complete**: If any two pieces of information do not outright contradict each other, they possess a join (least upper bound), allowing consistent computation streams to merge safely.

1.5 A caveat: topology gives way to recursion-theoretic bookkeeping, not to simplicity

Scott’s 1980 pivot (§1–3 above) trades point-set topology for order theory and finite tokens, and for most of the monograph that trade genuinely simplifies the formalization: neighbourhood systems, approximable maps, and the domain-equation solutions of Lectures I–VI are combinatorial objects with comparatively little topological machinery behind them. But it would be a mistake to read this as “the 1980 lectures are elementary throughout.” From Lecture VII onward, Scott’s subject is **computability in domains** — effectively given presentations, computable elements, computable maps — and this content is intrinsically recursion theory, not topology. Mechanizing it honestly means mechanizing recursion theory, and that turns out to be the more expensive

half of this formalization by a wide margin, for a reason specific to this project’s foundational choices rather than to Scott’s mathematics:

Mathlib’s own recursion theory (`Nat.Primrec`, `Primrec`, `Partrec`, `ComputablePred`) is unusable here because its correctness lemmas are proved via tactics (`grind`, `lia`, the `Nat.unpair_pair` simp set) that route through `Classical.choice`, which would silently taint every downstream computability result in this project with an axiom this formalization otherwise avoids (see the choice discipline below). The practical consequence is that `Recursive.lean` — a *from-scratch*, choice-free recursion theory (primitive-recursive `Nat.pair/unpair` round-trips, `Nat.sqrt` correctness, recursively-decidable predicates and their closure lemmas, course-of-values recursion via fuel-indexed tables) — is now **4,000+ lines**, roughly **9× the size of `Basic.lean`** (450 lines), the file containing the actual domain-theoretic definitions (neighbourhood systems, elements, subsystems) that this whole formalization is nominally *about*. A representative data point: Exercise 8.15’s “computable elements correspond to effectively presented domains” half needed perhaps 30–40 lines of genuine neighbourhood-system reasoning (an intersection dichotomy, a filter-closure invariant), sitting on top of 250+ lines encoding finite binary trees as `Nat.pair`-coded naturals and proving their evaluation is primitive recursive — a fact any computability textbook states in one sentence, made expensive here purely because Lean’s `Nat.pair/Nat.unpair` are not kernel-reducible and Mathlib’s off-the-shelf apparatus is off-limits by this project’s own discipline.

In short: the 1980 lectures did succeed in retiring topology as the dominant technical overhead, but Lecture VII onward replaces it with a different overhead — recursion-theoretic encoding and course-of-values bookkeeping — that is, in this formalization, actually the larger of the two by line count. This is a cost of the choice-free discipline adopted below, not a property of Scott’s mathematics; a formalization willing to inherit `Classical.choice` through Mathlib’s recursion theory would find Lecture VII’s exercises considerably shorter.

1.6 Chapter inclusion hierarchy (Lean imports)

Each node is a lecture (Scott chapter) at the point its Lean modules are first introduced. An edge $A \rightarrow B$ means some module belonging to lecture B has a direct `import` of a module belonging to lecture A (transitive imports through intermediate lectures are *not* drawn). Edge labels count those direct import sites. This is the dependency picture relevant to splitting the monograph into separate arXiv papers.

2 Methodology

This section records the proof-engineering conventions of the formalization—the parts of the development workflow that are of general academic interest, distilled from the project’s internal handoff notes.

2.1 Source material and inventory

The primary source is Dana Scott’s *Lectures on a Mathematical Theory of Computation* (Oxford, 1980; Technical Report PRG-19). OCR transcriptions live in `sources/PRG19_vision.md`; the structured inventory of every numbered Definition, Theorem, Example, and Exercise—with formalization status and proof notes—is maintained in this document (`arxiv.md`). Each item is keyed to Scott’s original numbering and cross-linked to its Lean module. Status values distinguish **Pass** (mechanized, builds green, zero `sorry`). Remaining mismatches versus the monograph are documented deferrals or expository items (Exercise 8.17 Part 2; Exercise 8.15 item

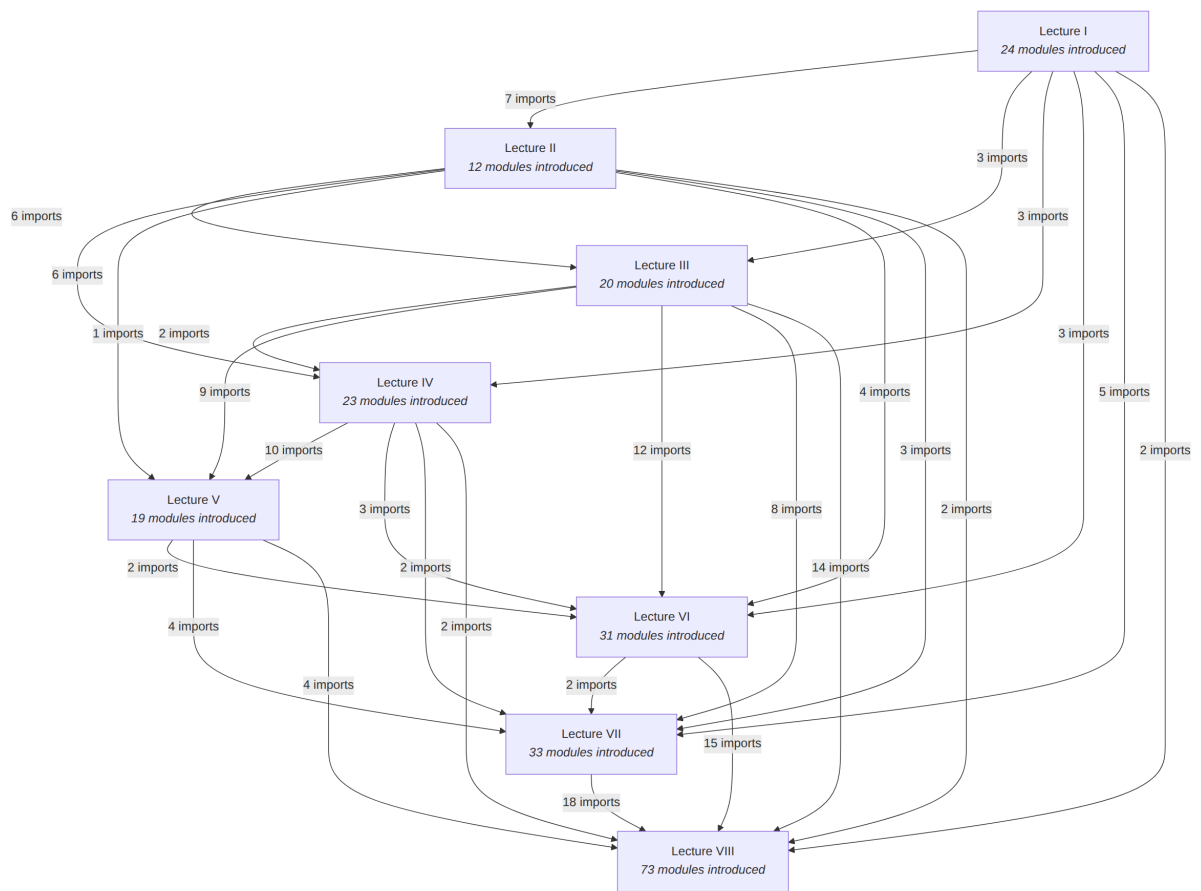


Figure 1: Chapter inclusion hierarchy (Lean imports).

6g; pictures / “discuss” prompts), not unfinished numbered proofs. The older **Partial** / **Not Yet** / **Deferred** legend described an earlier inventory state.

2.2 Neighbourhood systems as the uniform substrate

Following Scott’s 1980 pivot away from point-set topology, domains are encoded uniformly as **neighbourhood systems**: a master set Δ , a family $\mathcal{D}$ of neighbourhoods (filters on Δ), and domain elements as filters over $\mathcal{D}$. Approximable maps, products, function spaces, sums, and fixed-point combinators are built on this substrate in `Basic.lean`, `Approximable.lean`, `Product.lean`, and `FunctionSpace.lean`. Positive systems (Exercise 1.19) and effectively given presentations (Definition 7.1) are layered on top when Scott’s exercises demand computability content.

2.3 Custom recursion theory (Lecture VII)

For **effectively given** domains Scott requires two index relations to be *recursively decidable*: (i) intersection equality $X_n \cap X_m = X_k$, and (ii) consistency $\exists k. X_k \subseteq X_n \cap X_m$. Rather than mathlib’s `Computable/ComputablePred` development—which pulls `Classical.choice` through tactics such as `grind`, `lia`, and `Nat.unpair_pair`—we rebuilt the needed slice in `Recursive.lean`:

- `RecDecidable p :=  $\exists f, \text{Nat.Primrec } f \wedge \forall n, p\ n \leftrightarrow f\ n = 1$` (and the binary/ternary pair-codings `RecDecidable2`, `RecDecidable3`);
- choice-free correctness for `Nat.sqrt`, `Nat.pair/unpair`, and primitive-recursive `+/*`;
- closure lemmas (`RecDecidable.of_iff`, `.comp`, `.and`, `.or`, `.not`, bounded $\forall/\exists$ via `bForallFn/bExistsFn`);
- r.e. layers `REPred/REPred2` as projections of decidable relations.

Target axiom footprint for data constructions and core proofs: $\subseteq \{\text{propext}, \text{Quot.sound}\}$. `Classical.choice` is permitted only for genuinely unavoidable **Prop-level** steps (e.g. classical case splits on membership in an arbitrary system) and is always called out in proof notes. Each completed module is audited with `#print axioms`.

2.4 Incremental proof development

Large exercises are decomposed into small, revert-safe sessions rather than monolithic proofs. The inventory’s granularity — how far a single numbered item is split into lettered or numbered sub-goals before being discharged — was set by hand, session by session, rather than by a fixed rule. Some exercises (Exercise 7.22’s fifteen-plus sub-items above are the extreme case) were deliberately drilled down into small, independently provable pieces *before* any Lean was written, because the informal statement was large enough that a single monolithic proof attempt would have wasted effort on a failing whole-proof search. Others were left as a single inventory row even though the underlying Lean proof has considerable internal structure, because the statement was small enough that further decomposition would only add bookkeeping without changing how the proof was actually attacked. The deciding factors were pragmatic and varied by session: the size and combinatorial shape of the informal statement, the point at which further upfront planning showed diminishing returns, and how much compute budget seemed prudent to spend on decomposition versus a direct proof attempt. Consequently, the depth of decomposition visible in the inventory does not track a formal complexity metric of Scott’s original exercises; it is an artefact of session-by-session engineering judgment and should be read as such, not as a claim about which parts of PRG-19 are intrinsically harder to formalize.

3 Chronological Formalization Narrative

Below is the chronological narrative of the formalization, organized step-by-step using Dana Scott's original numbering system from the PRG-19 monograph.

3.1 Lecture I: Domains by Neighbourhoods

Definition 1.1

A neighbourhood system is a family $\mathcal{D}$ of subsets of a token set Δ , containing Δ and closed under intersection of finite consistent pairs. Lean keeps Δ as a field (**master**) rather than hard-wiring **Set.univ**, and records that Δ is non-empty (**master_nonempty**). Finite intersection follows Scott's recursive convention **interUpTo** (empty product is Δ ; the successor intersects the next set).

Lean 4 source

```
structure NeighborhoodSystem (a : Type*) where
  mem : Set a -> Prop
  master : Set a
  master_nonempty : master.Nonempty
  master_mem : mem master
  inter_mem : forall {X Y Z : Set a}, mem X -> mem Y -> mem Z ->
    Z subseteq X ∩ Y -> mem (X ∩ Y)
  sub_master : forall {X : Set a}, mem X -> X subseteq master
```

Factoid 1.1a

Empty intersection is Δ : **interUpTo_zero**.

Factoid 1.1b

Successor peels the last factor: **interUpTo_succ**.

Theorem 1.1c

A finite consistent sequence of neighbourhoods has its intersection again in $\mathcal{D}$.

Lean 4 source

```
theorem consistent_iff_interUpTo_mem (X : Nat -> Set a) {n : Nat}
  (hX : forall i, i < n -> V.mem (X i)) :
  V.Consistent X n <-> V.mem (V.interUpTo X n)
```

Example 1.2

Scott's first finite system: $\Delta = \{0, 1\}$ and $\mathcal{D} = \{\Delta, \{0\}, \{1\}\}$. Tokens 0 and 1 are completely specified and identified with the two total elements. The only partial element is $\perp = \{\Delta\}$: either no information, or a decision for 0 or 1. The three filters are **bot**, **elemZero**, and **elemOne**.

Lean 4 source

```
theorem element_classification (x : neighborhoodSystem.Element) :
  x = bot ∨ x = elemZero ∨ x = elemOne
```

Example 1.3

$\Delta = \{0, 1, 2\}$ with the nested chain $\mathcal{D} = \{\Delta, \{1, 2\}, \{2\}\}$. Approximation proceeds in two steps to the unique total (token 2), rather than in one jump. Tokens 0 and 1 are not total, and the direction of approximation is unique. The three filters form a linear chain $\perp \sqsubset \{1, 2\} \sqsubset \{2\}$.

Lean 4 source

```
theorem element_classification (x : neighborhoodSystem.Element) :
  x = bot ∨ x = elemTwelve ∨ x = elemTwo
```


Example 1.4

The depth-2 binary tree on tokens $\{\Lambda, 0, 1, 00, 01, 10, 11\}$, with neighbourhoods the cones of extensions above each node. The four top nodes 00, 01, 10, 11 are the totals; lower nodes are partial sequences. Branching records a choice of how to extend a partial sequence. Seven filters: $\perp$, two incomparable branch partials, and four total leaves.

Lean 4 source

```
theorem element_classification (x : neighborhoodSystem.Element) :
  x = bot  \ /  x = elemZero  \ /  x = elemOne  \ /
    x = elem00 \ /  x = elem01 \ /  x = elem10  \ /  x = elem11
```

Factoid 1.4a

Nested-or-disjoint families are neighbourhood systems: `NestedOrDisjoint`, `ofNestedOrDisjoint`.

Example 1.5

All nonempty subsets of $\Delta = \{0, 1, 2, 3\}$, a direct generalization of Example 1.2. Consistency is nonempty intersection, so neighbourhoods need not be nested or disjoint. Combinations of neighbourhoods are as varied as a four-element set allows.

Lean 4 source

```
def neighborhoodSystem : NeighborhoodSystem Token where
```

Factoid 1.5a

Sets are consistent iff they have nonempty intersection: `consistent_iff_inter_nonempty`.

Factoid 1.5b

Limit families of equivalent neighbourhood sequences coincide: `limitFamily_eq_iff`.

Definition 1.6

The (ideal) elements of $\mathcal{D}$ are the filters: subfamilies $x \setminus \text{subseq} \setminus \text{mathcal{D}}$ containing Δ , closed under $\cap$, and upward closed in $\mathcal{D}$. The domain $|\mathcal{D}|$ is the type of all such filters. Maximal filters are the total elements; partial filters are kept as first-class data.

Lean 4 source

```
structure Element where
  mem : Set a -> Prop
  sub : forall {X}, mem X -> V.mem X
  master_mem : mem V.master
  inter_mem : forall {X Y}, mem X -> mem Y -> mem (X ∩ Y)
  up_mem : forall {X Y}, mem X -> V.mem Y -> X subseq Y -> mem Y
```

Definition 1.7

For $X \setminus \text{in} \setminus \text{mathcal{D}}$ the principal filter $\uparrow X = \{Y \setminus \text{in} \setminus \text{mathcal{D}} \mid X \setminus \text{subseq} Y\}$ is the finite element determined by that neighbourhood. These are the elements given by a single neighbourhood rather than a limit of many.

Lean 4 source

```
def principal {X : Set a} (hX : V.mem X) : V.Element where
```

Factoid 1.7a

$X \setminus \text{mapsto} \setminus \text{uparrow} X$ is one-one and inclusion-reversing: `principal_le_iff`, `principal_injective`.

Factoid 1.7b

Every element is the union of its principal filters: `eq_iUnion_principal`.

Definition 1.8 (order)

Approximation on $|\mathcal{D}|$ is inclusion of filters: x approximates y iff $x \setminus \text{subteq} y$. This is the partial order on elements.

[Lean 4 source](#)

```
instance : PartialOrder V.Element where
```

Definition 1.8 ($\perp$, total)

Bottom is the least-defined element $\perp = \{\Delta\} = \uparrow \setminus \Delta$. Total elements are those maximal with respect to approximation.

[Lean 4 source](#)

```
def IsTotal (x : V.Element) : Prop := forall y, x <= y -> y <= x
```

Factoid 1.8a

$\perp$ approximates every element: `bot_le`.

Factoid 1.8b

A filter with a $\subseteq$ -minimum member X is $\uparrow X$: `eq_principal_of_isMin`.

Example 1.B

The infinite binary-tree system $B = \{\sigma \setminus \text{Sigma}^* \mid \setminus \sigma \setminus \text{in} \setminus \text{Sigma}^*\}$: neighbourhoods are cones of finite bit-strings, generalizing Example 1.4.

[Lean 4 source](#)

```
def B : NeighborhoodSystem Str :=
```

Exercise 1.B-sys

Show that B is a neighbourhood system: cones are pairwise nested-or-disjoint (`nestedOrDisjoint`).

[Lean 4 source](#)

```
theorem nestedOrDisjoint : NestedOrDisjoint memB
```

Exercise 1.B-elt

Show $\sigma x \setminus \text{in} B$ for every $x \setminus \text{in} B$: `sigmaElt`.

[Lean 4 source](#)

```
def sigmaElt (sigma : Str) (x : B.Element) : B.Element where
```

Factoid 1.B-mono

$\sigma_0 \setminus \text{perp} \setminus \text{subteq} \setminus \sigma_1 \setminus \text{perp}$ iff σ_0 is a prefix of σ_1 : `sigmaBot_le_iff`.

Factoid 1.B-lim

Every element is a union of finite approximations $\sigma_n \setminus \text{perp}$: `mem_iff_exists_sigmaBot`.

Definition 1.9

Two neighbourhood systems determine isomorphic domains when there is a one-one correspondence of $|\mathcal{D}_0|$ with $|\mathcal{D}_1|$ that preserves inclusion of elements both ways. Lean packages this as an order-isomorphism of `Element` types.

[Lean 4 source](#)

```
def Isomorphic {a b : Type*} (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : Prop
:=
  Nonempty (DomainIso V_0 V_1)
```

Theorem 1.10

The sets $[X] = \{x \setminus \text{mid} X \setminus \text{in} x\}$ form a neighbourhood system over $|\mathcal{D}|$ isomorphic to $\mathcal{D}$ itself.

Lean 4 source

```
theorem isomorphic_tokenSystem : V ~=? V.tokenSystem
```

Theorem 1.11

$|\mathcal{D}|$ is closed under countable intersections and under ascending countable unions; these are the GLB and LUB.

Lean 4 source

```
def iInter (x : Nat -> V.Element) : V.Element where
```

Exercise 1.12

Final segments of $\mathbb{N}$ as neighbourhoods: classify finite elements, the unique total, and every element as **fin n** or **top**.

Lean 4 source

```
def neighborhoodSystem : NeighborhoodSystem Nat :=
```

Exercise 1.13

Verify the claims about B and draw the binary tree with limit nodes along the top: each infinite path is a total element **branch**.

Lean 4 source

```
def branch (p : Nat -> Bool) : B.Element :=  
  B.iUnion (branchSeq p) (branchSeq_mono p)
```

Exercise 1.14

Finite nonempty subsets of $\mathbb{N}$ plus Δ form a neighbourhood system; totals are the singleton principals.

Lean 4 source

```
def neighborhoodSystem : NeighborhoodSystem Nat where
```

Exercise 1.15

Two infinite all-finite domains with no infinite strict chains, **flat** and **stem**, that are not isomorphic.

Lean 4 source

```
def flat : NeighborhoodSystem Nat :=
```

Exercise 1.16

Cofinite subsets of $\mathbb{N}$: $|\mathcal{D}|$ is isomorphic to $\mathcal{P}(\mathbb{N})$ under inclusion (**cofiniteIso**).

Lean 4 source

```
def cofiniteSystem : NeighborhoodSystem Nat where
```

Exercise 1.17

Rational open intervals on $\mathbb{R}$ plus Δ ; each real t determines a filter **filterAt t**.

Lean 4 source

```
def ratIntervalSystem : NeighborhoodSystem ? where
```

Exercise 1.18

A finitely consistent family of neighbourhoods has a least filter containing it; the intersection of any nonempty family of filters is a filter.

Lean 4 source

```
def FinitelyConsistent (C : Set (Set a)) : Prop :=
  forall (n : Nat) (X : Nat -> Set a), (forall i, i < n -> X i in C) -> V.Consistent
    X n
```

Exercise 1.19

A positive neighbourhood system (intersection is a neighbourhood iff nonempty) is a neighbourhood system; not every system is positive (`notPositiveSystem`).

Lean 4 source

```
def positiveExample : NeighborhoodSystem (Fin 2) :=
```

Exercise 1.20

The power system of down-sets $\uparrow X = \{Y \mid Y \subseteq D \mid Y \subseteq X\}$ is positive and isomorphic to $\mathcal{D}$ (`powerIso`).

Lean 4 source

```
def upSet (X : Set a) : Set (Set a)
```

Exercise 1.21

The token system $\{[X]\}$ over $|\mathcal{D}|$ is positive and complete: every filter is fixed by a unique point.

Lean 4 source

```
theorem tokenSystem_isPositive : V.tokenSystem.IsPositive
```

Exercise 1.22

The sets $[X]$ topologize $|\mathcal{D}|$: opens are the $\sqsubseteq$ -upper sets generated by basic neighbourhoods, and $\sqsubseteq$ is the specialization order.

Lean 4 source

```
instance instTopologicalSpaceElement : TopologicalSpace V.Element where
```

Exercise 1.23

A countable system admits a greedy total element `greedyElement`; every filter is determined by a sequence.

Lean 4 source

```
def greedyElement : V.Element where
```

Exercise 1.24

Using the Axiom of Choice, every element extends to a total element; the union of a nonempty chain of filters is a filter.

Lean 4 source

```
theorem exists_total_ge (x : V.Element) : exists t, V.IsTotal t /\ x <= t
```

Exercise 1.25

Nonempty final segments of a well-ordered Δ : $|\mathcal{D}|$ classifies nonempty lower sets; the unique total is not principal if Δ has no maximum.

Lean 4 source

```
def finalSegmentSystem (Delta : Type*) [LinearOrder Delta] [Nonempty Delta] :
  NeighborhoodSystem Delta :=
```

Exercise 1.26

For a commutative ring A , neighbourhoods $I(F) = \{G \mid F \subseteq G\}$ yield a domain isomorphic to the ideals of A .

Lean 4 source

```
def ringSystem : NeighborhoodSystem (Finset A) where
```

Exercise 1.27

A set of elements is bounded iff it has an upper bound; pairwise consistency of neighbourhoods matches boundedness of their principals.

Lean 4 source

```
def Bounded (X : Set V.Element) : Prop
```

3.2 Lecture II: Approximable Mappings

Definition 2.1

An approximable mapping $f : \mathcal{D}_0 \rightarrow \mathcal{D}_1$ is a relation between neighbourhoods: $\Delta_0 f \Delta_1$, closed under intersection on the output, and monotone in a sharper input and a blunter output. It is the neighbourhood-level data of a map of domains. Relation-extensionality is `ext`.

Lean 4 source

```
structure ApproximableMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) where
  rel : Set a -> Set b -> Prop
  rel_dom : forall {X Y}, rel X Y -> V_0.mem X
  rel_cod : forall {X Y}, rel X Y -> V_1.mem Y
  master_rel : rel V_0.master V_1.master
  inter_right : forall {X Y Y'}, rel X Y -> rel X Y' -> rel X (Y ∩ Y')
  mono : forall {X X' Y Y'}, rel X Y -> X' ⊆ X -> Y ⊆ Y' ->
    V_0.mem X' -> V_1.mem Y' -> rel X' Y'
```

Proposition 2.2

An approximable mapping determines an elementwise function $f : |\mathcal{D}_0| \rightarrow |\mathcal{D}_1|$ by $f(x) = \{Y \mid \exists X \text{ in } x, X f Y\}$.

Lean 4 source

```
def toElementMap (f : ApproximableMap V_0 V_1) (x : V_0.Element) : V_1.Element where
```

Example 2.3

Parity of the number of 0s before the first 1, as a map $B \rightarrow T$ into the two-token domain of Example 1.2. Outcomes are `true`, `false`, or $\perp$ when the prefix does not yet decide.

Lean 4 source

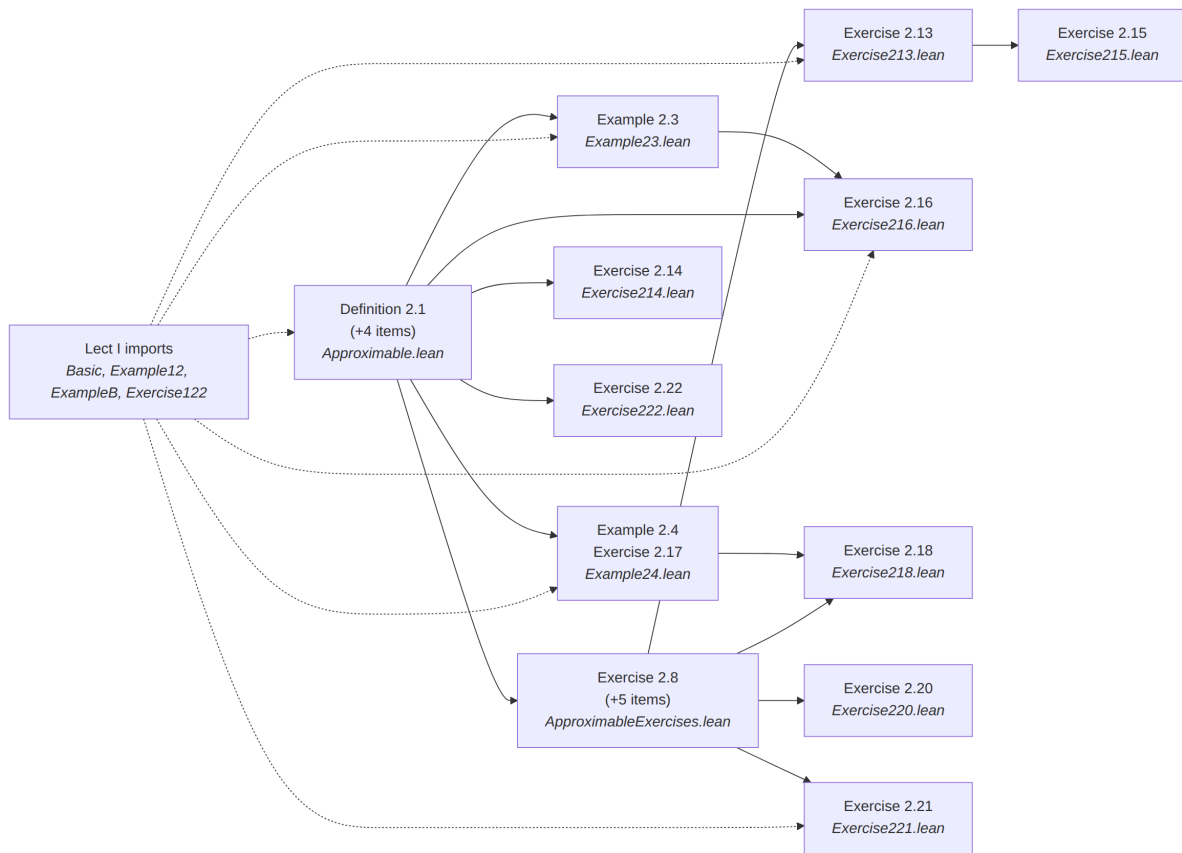


Figure 3: Lean module dependencies for Lecture II: Approximable Mappings.

```
def parityMap : ApproximableMap B T where
```

Example 2.4

The map $g : B \setminus to B$ that deletes the first consecutive run of 1s while copying the rest. The infinite sequence 1^∞ is sent to $\perp$.

Lean 4 source

```
def runMap : ApproximableMap B B where
```

Theorem 2.5

Neighbourhood systems and approximable mappings form a category, with identity $X I_D Y$ iff $X \setminus subseteq Y$ and composition by an intermediate neighbourhood.

Lean 4 source

```
def idMap (V : NeighborhoodSystem a) : ApproximableMap V V where
```

Proposition 2.6

The elementwise assignment is a functor: I_D acts as the identity and $(g \setminus circ f)(x) = g(f(x))$.

Lean 4 source

```
theorem toElementMap_comp (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1)
  (x : V_0.Element) :
  (g.comp f).toElementMap x = g.toElementMap (f.toElementMap x)
```

Theorem 2.7

Every domain isomorphism arises from an approximable mapping, and finite elements go to finite elements.

Lean 4 source

```
theorem exists_approximable_of_iso (e : V_0.Element ?o V_1.Element) :
  exists f : ApproximableMap V_0 V_1, forall x, f.toElementMap x = e x
```

Exercise 2.8

An approximable mapping is determined by its values on finite elements, and every monotone map of finite elements extends to one (`ofMono`).

Lean 4 source

```
theorem eq_of_toElementMap_principal {f g : ApproximableMap V_0 V_1}
  (h : forall (X : Set a) (hX : V_0.mem X),
    f.toElementMap (V_0.principal hX) = g.toElementMap (V_0.principal hX)) : f = g
```

Exercise 2.9

The elementwise map satisfies $f(x) = \bigcup \{f(\uparrow X) \setminus mid X \setminus in x\}$.

Lean 4 source

```
theorem toElementMap_mem_iff_principal (f : ApproximableMap V_0 V_1) (x : V_0.Element)
  {Y : Set b} :
  (f.toElementMap x).mem Y <->
  exists (X : Set a) (hx : x.mem X),
    (f.toElementMap (V_0.principal (x.sub hx))).mem Y
```

Exercise 2.10

The pointwise meet of two approximable maps is approximable: $h(x) = f(x) \setminus cap g(x)$.

Lean 4 source

```
def interMap (f g : ApproximableMap V_0 V_1) : ApproximableMap V_0 V_1 where
```

Exercise 2.11

The directed union of a family of elements is an element, and approximable maps preserve directed unions.

Lean 4 source

```
def iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I]
  (a : I -> V.Element) (hdir : forall i j, exists k, a i <= a k /\ a j <= a k) : V.
  Element :=
```

Exercise 2.12

The pointwise directed union of a family of approximable maps is approximable.

Lean 4 source

```
def iSupMap {I : Type*} [Nonempty I] (f : I -> ApproximableMap V_0 V_1)
  (hdir : forall i j, exists k, (forall X Y, (f i).rel X Y -> (f k).rel X Y) /\
    (forall X Y, (f j).rel X Y -> (f k).rel X Y)) : ApproximableMap V_0 V_1 where
```

Exercise 2.13

Approximable mappings are exactly the continuous maps between the $|\mathcal{D}|$ spaces of Exercise 1.22.

Lean 4 source

```
theorem continuous_toElementMap (f : ApproximableMap V_0 V_1) :
  Continuous (fun x => f.toElementMap x)
```

Exercise 2.14

A domain isomorphism e induces a neighbourhood correspondence φ with $e(\uparrow X) = \uparrow \varphi(X)$ and $X \text{ (ofIso } e) Y$ iff $\varphi(X) \setminus \text{subseq} Y$.

Lean 4 source

```
theorem phi_spec (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) :
  e (V_0.principal hX) = V_1.principal (phi_mem e hX)
```

Exercise 2.15

The one-token Sierpiński system O has elements $\perp \setminus \text{sqsubset} \setminus \text{top}$; open subsets of $|\mathcal{D}|$ correspond to approximable maps $\mathcal{D} \rightarrow O$.

Lean 4 source

```
def 0 : NeighborhoodSystem (Fin 1) :=
```

Exercise 2.16

The shift $x \setminus \text{mapsto} \sigma x$ on $|B|$ is approximable (`sigmaMap`). The parity map of Example 2.3 is the unique approximable map satisfying $f(1x) = \text{true}$, $f(01x) = \text{false}$, $f(00x) = f(x)$.

Lean 4 source

```
theorem toElementMap_sigmaMap (sigma : Str) (x : B.Element) :
  (sigmaMap sigma).toElementMap x = sigmaElt sigma x
```

Exercise 2.17

The run-deletion map of Example 2.4 is approximable and is uniquely determined by $g(0x) = 0g(x)$, $g(11x) = g(1x)$, $g(10x) = 0x$, and $g(1) = \perp$.

Lean 4 source

```
def runMap : ApproximableMap B B where
```

Exercise 2.18

The spacing map $h : B \rightarrow B$ ($b \mapsto \text{mapstob } b$) has a left inverse k but is not an isomorphism.

[Lean 4 source](#)

```
def hMap : ApproximableMap B B where
```

Exercise 2.19

A two-variable approximable mapping is a ternary neighbourhood relation with the natural generalizations of Definition 2.1.

[Lean 4 source](#)

```
structure ApproximableMap_2 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
  (V_2 : NeighborhoodSystem c) where
  rel : Set a -> Set b -> Set c -> Prop
  rel_dom_0 : forall {X Y Z}, rel X Y Z -> V_0.mem X
  rel_dom_1 : forall {X Y Z}, rel X Y Z -> V_1.mem Y
  rel_cod : forall {X Y Z}, rel X Y Z -> V_2.mem Z
  master_rel : rel V_0.master V_1.master V_2.master
  inter_right : forall {X Y Z Z'}, rel X Y Z -> rel X Y Z' -> rel X Y (Z ∪ Z')
  mono : forall {X X' Y Y' Z Z'}, rel X Y Z -> X' ⊆ X -> Y' ⊆ Y -> Z
    ⊆ Z' ->
    V_0.mem X' -> V_1.mem Y' -> V_2.mem Z' -> rel X' Y' Z'
```

Exercise 2.20

The powerset domain $\mathcal{P}$ of cofinite neighbourhoods on $\mathbb{N}$ has $|\mathcal{P}| \simeq |\mathcal{P}(\mathbb{N})|$; union, intersection, successor, and predecessor are approximable.

[Lean 4 source](#)

```
def equivSetNat : powerSet.Element ?o Set Nat where
```

Exercise 2.21

A system $C \sqsubseteq^* B$ in which both finite and infinite sequences are total, together with left-biased juxtaposition xy .

[Lean 4 source](#)

```
def C : NeighborhoodSystem Str :=
```

Exercise 2.22

Any family of sets closed under nonempty intersections and directed unions is inclusion-isomorphic to a domain.

[Lean 4 source](#)

```
def reprIso : (reprSystem C hInter hne).Element ?o {X : Set tau // X in C} where
```

3.3 Lecture III: Domain Constructs

Definition 3.1

Given neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ over disjoint token sets, the *product* is $\{X \sqcup Y \mid X \in \mathcal{D}_0, Y \in \mathcal{D}_1\}$. Lean tags tokens by $\alpha \oplus \beta$, so a product

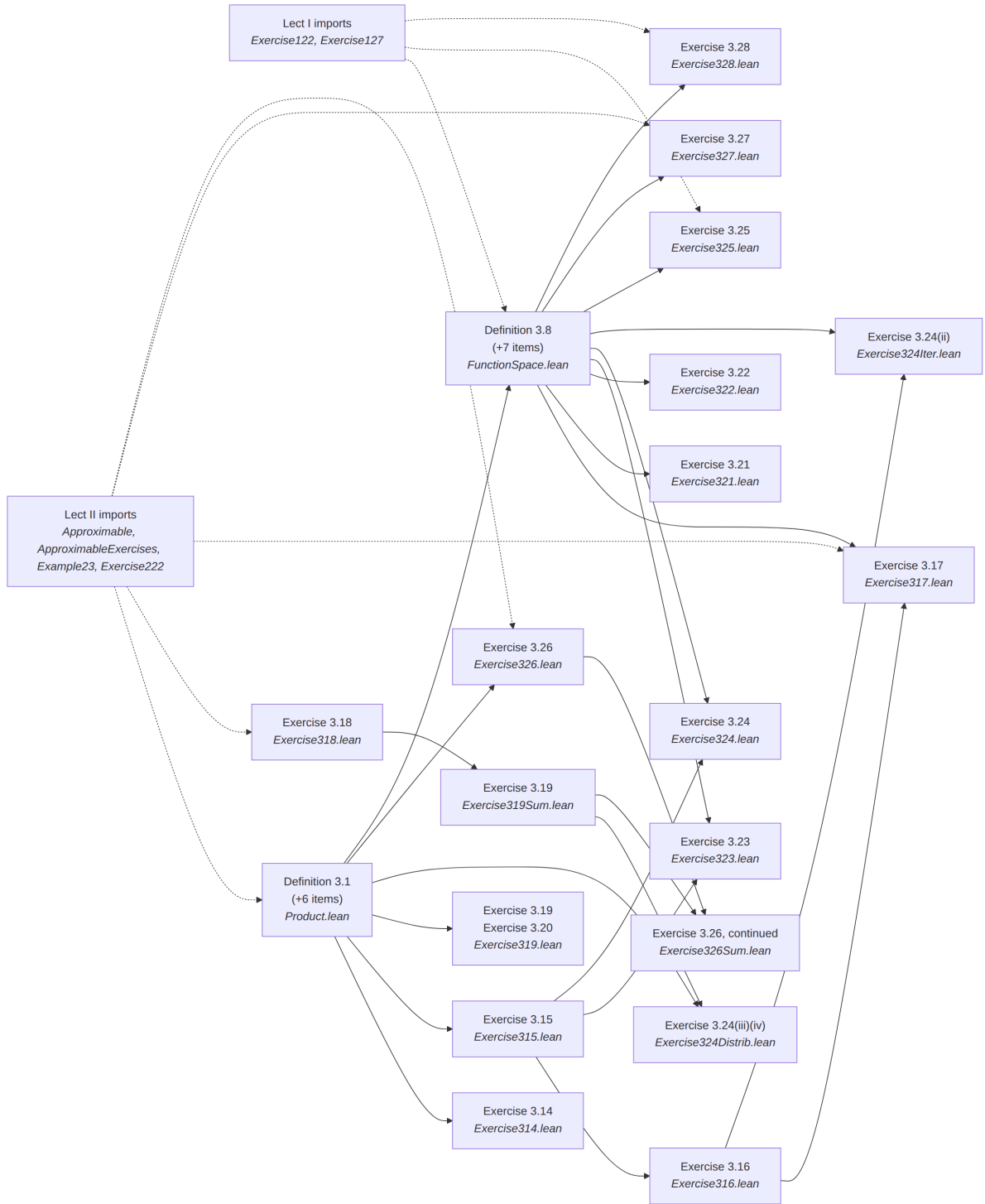


Figure 4: Lean module dependencies for Lecture III: Domain Constructs.

neighbourhood is $\text{prodNbhd } X \ Y = \text{Sum.inl } '' \ X \cup \text{Sum.inr } '' \ Y$. Elements pair as $\langle x, y \rangle \setminus \text{rangle} = \{X \setminus \text{cup} Y \setminus \text{mid} X \setminus \text{inx}, Y \setminus \text{iny}\}$, recovered by `Element.fst/snd`.

Lean 4 source

```
def prod (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : NeighborhoodSystem (a ? b)
  where
```

Proposition 3.2

The product is a neighbourhood system, and every element is uniquely a pair, order-isomorphically.

Lean 4 source

```
def prodEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
  (prod V_0 V_1).Element ?o V_0.Element x V_1.Element where
```

Definition 3.3

Projections p_0, p_1 relate a product neighbourhood to a factor by inclusion of that component. The paired map $\langle f, g \rangle \setminus \text{rangle}$ relates Z to $X \setminus \text{cup} Y$ just when $Z \text{ } f \ X$ and $Z \text{ } g \ Y$.

Lean 4 source

```
def paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) :
  ApproximableMap V_2 (prod V_0 V_1) where
```

Proposition 3.4

The projections and pairing are approximable, with $p_i \setminus \text{circ} \setminus \text{langle} f, g \setminus \text{rangle}$ recovering f or g , $h = \langle p_0 \setminus \text{circ}, p_1 \setminus \text{circ} \rangle \setminus \text{rangle}$, and $\langle f, g \rangle \setminus \text{rangle}(w) = \langle f(w), g(w) \rangle \setminus \text{rangle}$.

Lean 4 source

```
theorem proj_0_comp_paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) :
  (proj_0 V_0 V_1).comp (paired f g) = f :=
```

Theorem 3.5

A two-argument elementwise map comes from an approximable mapping iff it is separately approximable in each variable.

Lean 4 source

```
def map_2Equiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
  NeighborhoodSystem ?) :
  ApproximableMap (prod V_0 V_1) V_2 ? ApproximableMap_2 V_0 V_1 V_2 where
```

Lemma 3.6

The constant function with value b comes from the relation $X \text{ } b \ Y$ iff $Y \setminus \text{in} b$.

Lean 4 source

```
def constMap (V_0 : NeighborhoodSystem a) (b : V_1.Element) : ApproximableMap V_0 V_1 where
```

Proposition 3.7

Multivariate approximable maps are closed under substitution, assembled from pairing and composition.

Lean 4 source

```
theorem substitution_toElementMap (F : ApproximableMap (prod V_0 V_1) V_2)
  (a : ApproximableMap V_3 V_0) (b : ApproximableMap V_3 V_1) (w : V_3.Element) :
```

Definition 3.8

The *function space* $(\mathcal{D}_0 \setminus to \mathcal{D}_1)$ has tokens the approximable maps and neighbourhoods the finite nonempty intersections of *step* sets $[X, Y] = \{f \setminus mid X f Y\}$. Lean records a single step as **step** and a finite intersection as **stepFun**.

Lean 4 source

```
def funSpace (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
  NeighborhoodSystem (ApproximableMap V_0 V_1) where
```

Proposition 3.9

A finite family of steps is consistent iff whenever a subcollection of the X_i is consistent, so is the matching Y_i ; the least map in the intersection satisfies $X f_0 Y$ iff $\bigcap \{Y_i \setminus mid X \setminus subseteq X_i\} \subseteq Y$.

Lean 4 source

```
def leastMap (L : List (Set a x Set b)) (_hL : forall p in L, V_0.mem p.1 /\ V_1.mem p
  .2)
  (hcons : forall {X}, V_0.mem X -> V_1.mem (interYs V_1.master L X)) : ApproximableMap
  V_0 V_1 where
```

Theorem 3.10

Every filter in the function space is fixed by a unique approximable mapping.

Lean 4 source

```
def funSpaceEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
  (funSpace V_0 V_1).Element ?o ApproximableMap V_0 V_1 where
```

Theorem 3.11

Evaluation $\text{eval} : (\mathcal{D}_1 \setminus to \mathcal{D}_2) \setminus times \mathcal{D}_1 \setminus to \mathcal{D}_2$ is approximable and satisfies $\text{eval}(f, x) = f(x)$.

Lean 4 source

```
def evalMap (V_1 : NeighborhoodSystem b) (V_2 : NeighborhoodSystem ?) :
  ApproximableMap (prod (funSpace V_1 V_2) V_1) V_2 :=
```

Theorem 3.12

Currying is an order-isomorphism $|\mathcal{D}_0 \setminus times \mathcal{D}_1 \setminus to \mathcal{D}_2| \simeq |\mathcal{D}_0 \setminus to (\mathcal{D}_1 \setminus to \mathcal{D}_2)|$, with $\text{eval} \circ \langle \text{curry}(g) \rangle_{p_0, p_1} = g$.

Lean 4 source

```
def curry (g : ApproximableMap (prod V_0 V_1) V_2) : ApproximableMap V_0 (funSpace V_1 V_2)
  where
```

Theorem 3.13(i)

Approximable maps are ordered pointwise: $f \setminus sqsubseteq g$ iff $f(x) \setminus sqsubseteq g(x)$ for all x .

Lean 4 source

```
theorem le_iff_toElementMap_le {f g : ApproximableMap V_0 V_1} :
  f <= g <-> forall x, f.toElementMap x <= g.toElementMap x :=
```

Theorem 3.13(ii)

A family of maps is bounded iff the pointwise values are bounded at every argument.

Lean 4 source

theorem mapsBounded_iff_pointwiseBounded {F : Set (ApproximableMap V_0 V_1)} :

Theorem 3.13(iii)

When F is bounded, $(\bigsqcup F)(x) = \bigsqcup \{f(x) \mid f \in F\}$.

Lean 4 source

```
def sSupMaps (F : Set (ApproximableMap V_0 V_1)) (hF : PointwiseBounded F) : ApproximableMap
  V_0 V_1 :=
```

Exercise 3.14

Disjointness of token sets is unnecessary: tag copies as $0 \setminus \Delta_0 \cup 1 \setminus \Delta_1$ (already $\alpha \oplus \beta$). The diagonal $\text{diag} : \mathcal{D} \rightarrow \mathcal{D} \times \mathcal{D}$ satisfies $\text{diag}(x) = \langle x, x \rangle$.

Lean 4 source

```
def diag : ApproximableMap V (prod V V) := paired (idMap V) (idMap V)
```

Exercise 3.15

The usual product isomorphisms: commutativity, associativity, the empty (unit) product, and functoriality in each factor.

Lean 4 source

```
theorem prod_comm_isomorphic : prod V_0 V_1 ~=? prod V_1 V_0
```

- `prod_assoc_isomorphic` — $\mathcal{D}_0 \times (\mathcal{D}_1 \times \mathcal{D}_2) \cong (\mathcal{D}_0 \times \mathcal{D}_1) \times \mathcal{D}_2$.
- `prod_unit_isomorphic` — $\mathcal{D}_0 \times 1 \cong \mathcal{D}_0$.
- `Isomorphic.prod` — $\mathcal{D}_0 \times \mathcal{D}'_0$ and $\mathcal{D}_1 \times \mathcal{D}'_1$ imply $\mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}'_0 \times \mathcal{D}'_1$.

Exercise 3.16

The infinite product $\mathcal{D}^\infty$ over tagged copies Δ^∞ is a neighbourhood system, isomorphic to $\mathcal{D} \times \mathcal{D}^\infty$, with elements the sequences of $|\mathcal{D}|$ -elements.

Lean 4 source

```
def iterSys (V : NeighborhoodSystem a) : NeighborhoodSystem (Nat x a) where
```

Exercise 3.17

There is a one-one approximable $f : \mathcal{B} \rightarrow \mathcal{T}^\infty$ with a retraction g , so $g \circ f = I_{\mathcal{B}}$ and $f \circ g \sqsubseteq I_{\mathcal{T}^\infty}$.

Lean 4 source

```
theorem gf_eq_id : g.comp f = idMap B :=
```

Exercise 3.18

The separated sum $\mathcal{D}_0 + \mathcal{D}_1$ adjoins a tag Λ and the two injections; $\text{out}_i \circ \text{in}_i = I$, provided no neighbourhood is empty.

Lean 4 source

```
def sum (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
  (h_0 : forall X, V_0.mem X -> X.Nonempty) (h_1 : forall Y, V_1.mem Y -> Y.Nonempty) :
  NeighborhoodSystem (Option (a ? b)) where
```

Exercise 3.19

Functorial $f \setminus times g$ and $f + g$ on products and sums, with $(f \setminus times g)(x, y) = \langle f(x), g(y) \setminus rangle$ and $out_i \setminus circ(f + g) \setminus circ \setminus mathrm{min}_i$ recovering f or g .

Lean 4 source

```
def prodMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1') :
  ApproximableMap (prod V_0 V_1) (prod V_0' V_1') :=
```

- `sumMap` — the sum combinator on $\mathcal{D}_0 + \mathcal{D}_1$.
- `outMap0_comp_sumMap_comp_inMap0` — $out_0 \setminus circ(f + g) \setminus circ \setminus mathrm{min}_0 = f$.

Exercise 3.20

On the category of domains, $+$ and $\times$ are functors, and $\times$ is the categorical product.

Lean 4 source

```
theorem paired_unique (h_0 : ApproximableMap V_2 V_0) (h_1 : ApproximableMap V_2 V_1)
  (k : ApproximableMap V_2 (prod V_0 V_1)) (hk_0 : (proj_0 V_0 V_1).comp k = h_0)
  (hk_1 : (proj_1 V_0 V_1).comp k = h_1) : k = paired h_0 h_1 :=
```

Exercise 3.21

A step $[Y, Z]$ determines Y and Z uniquely when $Z \setminus neq \setminus Delta_2$; when $Z = \Delta_2$ the curry biconditional still holds.

Lean 4 source

```
theorem step_eq_of_ne_master {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z)
```

Exercise 3.22

Composition is an approximable map of function spaces, with $\text{comp}(g, f) = g \setminus circ f$.

Lean 4 source

```
def compMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 : NeighborhoodSystem
  ?) :
  ApproximableMap (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) (funSpace V_0 V_2) :=
```

Exercise 3.23

Theorems 3.11–3.12 make domains and approximable maps a cartesian closed category; the terminal object is the one-neighbourhood system.

Lean 4 source

```
def homAdjunction (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
  (V_2 : NeighborhoodSystem ?) :
  ApproximableMap (prod V_0 V_1) V_2 ?o ApproximableMap V_0 (funSpace V_1 V_2) :=
```

Exercise 3.24

Further function-space isomorphisms, and mapping relationships where the separated-sum bottom blocks a true iso.

Lean 4 source

```
theorem funProd_isomorphic :
  funSpace V_0 (prod V_1 V_2) ~=? prod (funSpace V_0 V_1) (funSpace V_0 V_2) :=
```

- `funIter_isomorphic` — $(\mathcal{D}_0 \setminus to \setminus mathcal{D}_1^\infty) \setminus cong(\mathcal{D}_0 \setminus to \setminus mathcal{D}_1)^\infty$.

- `distribMap` — a map $\mathcal{D}_0 \setminus \text{times}(\mathcal{D}_1 + \mathcal{D}_2) \setminus \text{to}(\mathcal{D}_0 \setminus \text{times} \setminus \text{mathcal{D}_1}) + (\mathcal{D}_0 \setminus \text{times} \setminus \text{mathcal{D}_2})$.
- `copair` — the mediating map for $(\mathcal{D}_0 + \mathcal{D}_1) \setminus \text{to} \setminus \text{mathcal{D}_2}$.

Exercise 3.25

Open subsets of $|\mathcal{D}|$ form a domain, order-isomorphic to the maps into the Sierpiński space.

Lean 4 source

```
def openIso (V : NeighborhoodSystem a) :
  {U : Set V.Element // IsOpen U} ?o ApproximableMap V sierpinski where
```

Exercise 3.26

The conditional `cond :  $T \setminus \text{times} D \setminus \text{times} D \setminus \text{to} D$` selects the true or false branch and sends $\perp$ to $\perp$. A sum-valued variant and a tag map which recover the injection.

Lean 4 source

```
def cond (V : NeighborhoodSystem a) : ApproximableMap (prod TD (prod V V)) V where
```

- `condSum` — $T \setminus \text{times} D_0 \setminus \text{times} D_1 \setminus \text{to} D_0 + D_1$.
- `whichMap` — $D_0 + D_1 \setminus \text{to} T$.
- `cond_which` — $\text{cond}(\text{which } x, \text{in}_0(\text{out}_0 x), \text{in}_1(\text{out}_1 x)) = x$.

Exercise 3.27

Graphs of approximable maps form a complete lattice of subsets in the sense of Exercise 2.22, giving another proof that $\mathcal{D}_0 \setminus \text{to} \setminus \text{mathcal{D}_1}$ is a domain.

Lean 4 source

```
def graph (f : ApproximableMap V_0 V_1) : Set (Set a x Set b) := {p | f.rel p.1 p.2}
```

Exercise 3.28

Elementwise, the least map of $\bigcap [X_i, Y_i]$ is $f_0(x) = \bigsqcup \{\uparrow Y_i \setminus \text{mid } x \setminus \text{in}[X_i]\}$.

Lean 4 source

```
theorem toElementMap_leastMap_eq_sSup (L : List (Set a x Set b))
```

3.4 Lecture IV: Fixed Points and Recursion

Theorem 4.1

Every approximable $f : D \setminus \text{to} D$ has a least fixed point, constructed as the neighbourhoods reachable from Δ by a finite f -chain.

Lean 4 source

```
theorem toElementMap_fixElement (f : ApproximableMap V V) :
  f.toElementMap f.fixElement = f.fixElement :=
```

Theorem 4.2

The operator $\text{fix} : (D \setminus \text{to} D) \setminus \text{to} D$ is itself approximable, and $\text{fix}(f) = \bigsqcup_n f^n(\perp)$.

Lean 4 source

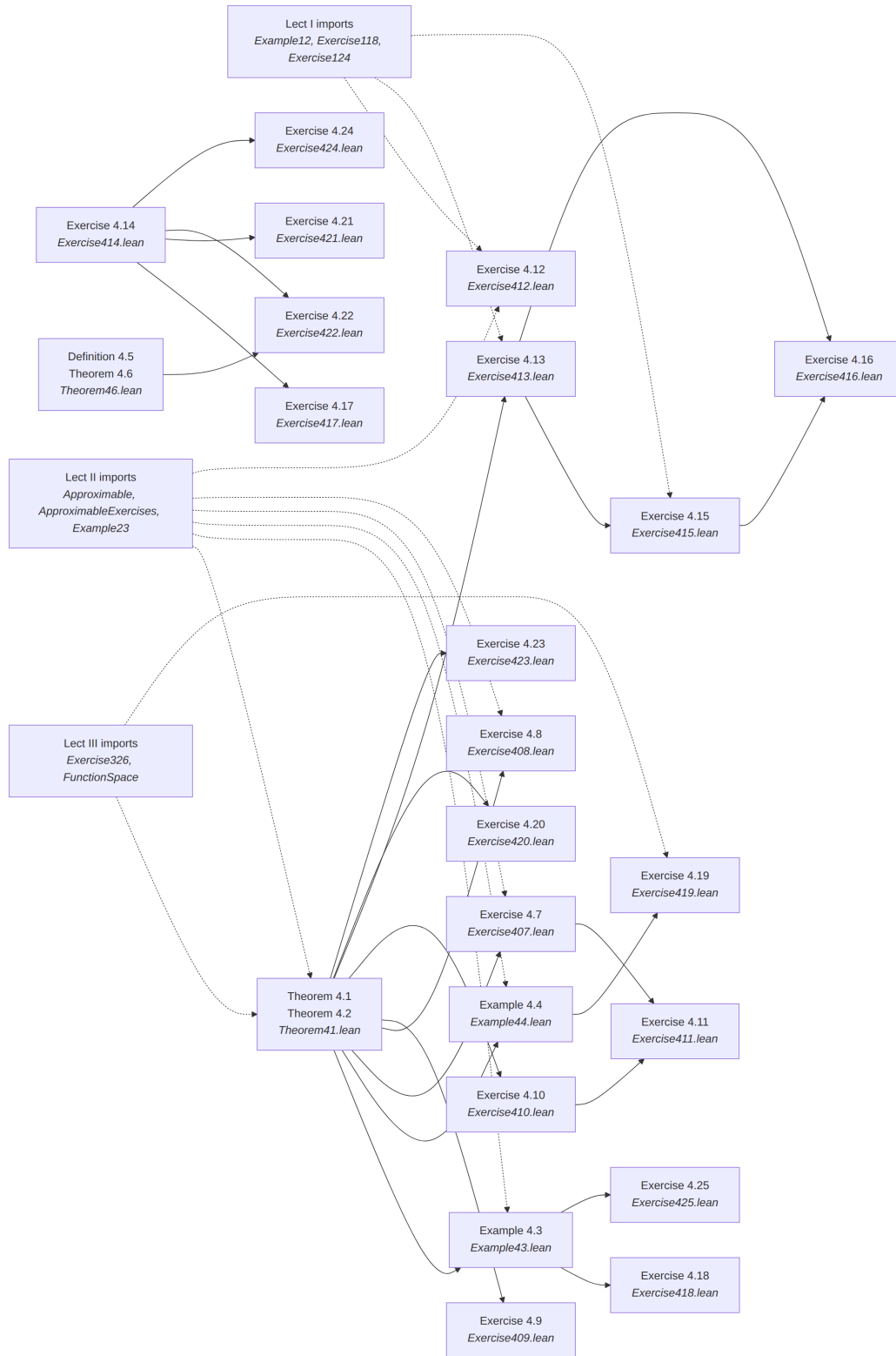


Figure 5: Lean module dependencies for Lecture IV: Fixed Points and Recursion.

```
def fixMap (V : NeighborhoodSystem a) : ApproximableMap (funSpace V V) V :=
```

Example 4.3

The flat naturals $N = \{\{n\} \mid n \in \mathbf{N}\} \cup \{\mathbf{N}\}$ generalize Example 1.2. Total elements are the principal filters $\hat{n}$; succ, pred, and zero : $N \rightarrow T$ equip N as a structured domain.

Lean 4 source

```
abbrev N : NeighborhoodSystem Nat :=
```

Example 4.4

The domain C of finite or infinite binary sequences has neighbourhoods the cones $\sigma \backslash \Sigma^*$ and the singletons $\{\sigma\}$. Cons, tail, and empty/zero/one : $C \rightarrow T$ make C a structured generalization of N ; $a = 01a$ is the alternating stream.

Lean 4 source

```
abbrev C : NeighborhoodSystem Str :=
```

Definition 4.5

A model for Peano's Axioms is a structured set $\langle \mathbf{N}, 0, + \rangle$ with 0 not a successor, successor injective, and induction: any subset containing 0 and closed under $+$ is the whole set.

Lean 4 source

```
structure PeanoModel (N : Type u) where
  zero : N
  succ : N -> N
  zero_ne_succ : forall n, zero != succ n
  succ_injective : Function.Injective succ
  induction : forall (s : Set N), zero in s -> (forall n in s, succ n in s) ->
    forall n, n in s
```

Theorem 4.6

Any two Peano models are isomorphic: the least relation containing $(0, \square)$ and closed under successors is a bijection.

Lean 4 source

```
theorem peano_models_isomorphic (P : PeanoModel M) (Q : PeanoModel N) :
  exists e : M ? N, e P.zero = Q.zero /\ forall m, e (P.succ m) = Q.succ (e m) :=
```

Exercise 4.7

If $a \sqsubset f(a)$, the chain $a \sqsubset f(a) \sqsubset f^2(a) \sqsubset \dots$ has a least upper bound that is a fixed point above a .

Lean 4 source

```
theorem fixAbove_isFixed (f : ApproximableMap V V) {a : V.Element} (ha : a <= f.toElementMap a) :
```

Exercise 4.8

A set containing $\perp$, closed under f , and closed under increasing ω -chains contains $\text{fix}(f)$ (fixed-point induction).

Lean 4 source

```
theorem fix_induction (f : ApproximableMap V V) (P : V.Element -> Prop)
  (hbot : P V.bot)
  (hstep : forall x, P x -> P (f.toElementMap x))
```

```
(hsup : forall (s : Nat -> V.Element) (hmono : Monotone s), (forall n, P (s n)) -> P
(supChain s hmono)) :
```

Exercise 4.9

The operator $\Psi(\theta)(f) = f(\theta(f))$ on $((D \setminus \text{to}D) \setminus \text{to}D)$ is approximable, and fix is its least fixed point.

Lean 4 source

```
def bigPsi (V : NeighborhoodSystem a) :
  ApproximableMap (funSpace (funSpace V V) V) (funSpace (funSpace V V) V) :=
```

Exercise 4.10

The subsystem D_a of elements below a is a domain; restricting f at a fixed point leaves a unique fixed point.

Lean 4 source

```
def relIso (a : V.Element) : (relSystem a).Element ?o {x : V.Element // x <= a} where
```

Exercise 4.11

The assignment $D \setminus \text{mapsto} \setminus \text{mathrm{fix}}_D$ is the unique family of fixed-point operators that is uniform for strict intertwining maps.

Lean 4 source

```
theorem fix_unique_of_uniform
```

Exercise 4.12

An approximable self-map need not have a greatest fixed point: the identity on T has two incomparable totals.

Lean 4 source

```
theorem no_greatest_fixedPoint :
```

Exercise 4.13

The apparent circularity between 4.1 (using $\mathbf{N}$) and 4.6 (using 4.1) is removed by taking the meet of pre-fixed points for monotone maps, then defining primitive recursion on $\mathbf{N}$.

Lean 4 source

```
def monoFix (f : V.Element -> V.Element) {a : V.Element} (ha : f a <= a) : V.Element :=
```

- `exists_unique_nat_rec` — unique $s : \mathbf{N} \rightarrow Z$ with $s(0) = z$ and $s(n^+) = (sn) \setminus \text{cdot}$.

Exercise 4.14

A monotone operator on PA always has a greatest fixed point, the union of all post-fixed sets.

Lean 4 source

```
def gfpSet (f : Set A -> Set A) : Set A := {a | exists x : Set A, x subseq f x /\ a
in x}
```

Exercise 4.15

A monotone self-map of a domain has a maximal fixed point (Zorn on post-fixed points) and a least one (`monoFix`).

Lean 4 source

```
theorem exists_maximal_fixedPoint (f : V.Element -> V.Element) (hf : Monotone f) :
```

Exercise 4.16

The optimal fixed point is the least fixed point over the meet of a nonempty family of fixed points; it lies below every member and is consistent with every fixed point.

Lean 4 source

```
def optimalFix (f : V.Element -> V.Element) (hf : Monotone f) (S : Set V.Element) (hS : S.
  Nonempty)
```

Exercise 4.17

In the powerset of a monoid, the least solution of $x = \{1\} \cup \{a, b\} \cup x \cdot x$ is the submonoid generated by $\{a, b\}$; `Set.univ` is another fixed point.

Lean 4 source

```
theorem lfpSet_eq_closure (a b : S) :
```

Exercise 4.18

The unproved claims of Example 4.3 hold: N has no empty neighbourhood, its elements are $\perp$ or $\hat{n}$, and `succ` is injective on totals.

Lean 4 source

```
theorem element_classification (x : N.Element) :
  x = N.bot \ / exists n, x = natElem n :=
```

Exercise 4.19

Strings $\{0, 1\}^*$ satisfy Peano-style axioms; `one : C \ toT` is recovered from `empty/zero/tail` by a fixed-point equation.

Lean 4 source

```
def oneMap : ApproximableMap C T :=
```

Exercise 4.20

Least fixed points swap under composition: $\text{fix}(f \circ g) = f(\text{fix}(g \circ f))$.

Lean 4 source

```
theorem fixElement_comp_comm (f g : ApproximableMap V V) :
  (f.comp g).fixElement = f.toElementMap (g.comp f).fixElement :=
```

Exercise 4.21

The order $\leq$ on $\text{subsetseq } \mathbf{N} \times \mathbf{N}$ is the unique fixed point of $\ell \mapsto \{(n, n)\} \cup \{(n, m^+) \mid \text{mid}(n, m) \mid \text{in } \ell\}$. The same recursion gives the up-sets $[m]$, addition, and multiplication.

Lean 4 source

```
theorem leRel_isFixed : leOp leRel = leRel
```

- `upSet` — $[m] = \{k \mid \text{mid } m \mid \text{le } k\}$.
- `addIso` — $\mathbf{N} \simeq [m]$ via $n \mapsto n + m$.
- `mulOp_lfp_eq_multiples` — least solution of $n \cdot \mathbf{N} = \{0\} \cup \{n + m \mid \text{mid } m \mid \text{in } n \cdot \mathbf{N}\}$.

Exercise 4.22

From axioms (i)(ii) alone, the least set containing 0 and closed under $+$ is a Peano model.

Lean 4 source

```
def nats : Set M
```

Exercise 4.23

If approximants a_n start at $\perp$, increase to the identity, and commute with f as $a_{n+1} \circ f = a_n \circ f \setminus \text{circa}_n$, then f has a unique fixed point.

Lean 4 source

```
theorem f_unique_fixedPoint (f : ApproximableMap V V) (a : Nat -> ApproximableMap V V)
```

Exercise 4.24

Schröder–Bernstein: the least fixed point of $T(X) = (A - g(B)) \setminus \text{cupg}(f(X))$ splits A into the two restrictions of a bijection.

Lean 4 source

```
theorem schroeder_bernstein (hf : Injective f) (hg : Injective g) :  
  exists h : A -> B, Bijective h :=
```

Exercise 4.25

The unary system C_1 on $\{1\}^*$ has nested tails and singletons, a successor `consMap`, and an infinite fixed point 1^∞ distinguishing it from flat N .

Lean 4 source

```
def C1 : NeighborhoodSystem Nat :=
```

3.5 Lecture V: Typed λ -Calculus

Theorem 5.1

every typed λ -term defines an approximable function of its free variables.

Lean 4 source

```
theorem var_fst (x : V_0.Element) (y : V_1.Element) :
```

Theorem 5.2

the conversion/substitution equation for suitably typed λ -terms.

Lean 4 source

```
theorem subst_value (g : ApproximableMap (prod V_0 V_1) V_2) (v : V_0.Element) (x : V_1.Element  
  ) :
```

Proposition 5.3

least fixed point of a pair-valued λ , coordinatewise (Bekić).

Lean 4 source

```
theorem sectionX_apply (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) (x : V_0.  
  Element) :
```


Proposition 5.4

fixed-point equation for $g: (D \rightarrow D)$.

Lean 4 source

```
theorem pfix_isFixed (tau : ApproximableMap (prod V V) V) (x : V.Element) :
```

Table 5.5

Combinators written as λ -terms: pairing, diagonal, swap, eval, const, curry, composition, and fix. Each is an approximable map on the product and function-space constructs of Lectures III–IV.

Lean 4 source

```
def pairC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
```

Theorem 5.6

every partial recursive $h: \mathbb{N} \rightarrow \mathbb{N}$ is λ -definable from the primitives `cond`, `succ`, `pred`, `zero`, and `0`.

Lean 4 source

```
theorem partrec_lamDef {n : Nat} {f : List.Vector Nat n -> . Nat} (h : Nat.Partrec' f) :
```

Exercise 5.7

Multi-variable λ and application recover from the one-variable forms via `p0`, `p1`, and pairing.

Lean 4 source

```
theorem surjective_pairing (z : (prod V_0 V_1).Element) :
```

Exercise 5.8

Bracket abstraction in I , K , S eliminates λ , leaving only application $\sigma(\tau)$.

Lean 4 source

```
def bracket {X : Dom} : {A : Dom} -> Poly X A -> CL (X.arrow A)
```

Exercise 5.9

Commuting approximable maps have a least common fixed point.

Lean 4 source

```
theorem commuting_least_common_fixed {f g : ApproximableMap V V} (hcomm : f.comp g = g.comp f)
:
```

Exercise 5.10

The smash product $D_0 \otimes D_1$ and the strict function space $D_0 \rightarrow \perp D_1$ satisfy the adjunction $(D_0 \otimes D_1) \rightarrow \perp D_2 \simeq D_0 \rightarrow \perp (D_1 \rightarrow \perp D_2)$.

Lean 4 source

```
def smash (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : NeighborhoodSystem (a ? b
) where
```

Exercise 5.11

D^∞ as bottomless stacks, with combinators `head`, `tail`, `push`, `diag`, and `map`.

Lean 4 source

```
def head : ApproximableMap (iterSys V) V := (proj_0 V (iterSys V)).comp (unfold V)
```

Exercise 5.12

The `while` combinator on D as a least fixed point.

Lean 4 source

```
def whileMap : ApproximableMap V V := toApproxMap (whileFix V p f)
```

Exercise 5.13

A one-one pairing $\text{num} : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ by the Cantor diagonal, inverted by `unnum` as `numEquiv`.

Lean 4 source

```
theorem num_injective : Function.Injective numP
```

Exercise 5.14

Approximable `fun` and `graph` on $P\omega$: `FunGraph` is the identity on continuous maps, and `graphFun` contains the identity.

Lean 4 source

```
def Fun (u : Set Nat) (x : Set Nat) : Set Nat :=
```

Exercise 5.15

The powerset of a monoid is a Kleene algebra; $\text{star } z = \bigcup_n z^n$ satisfies Arden's rule.

Lean 4 source

```
def star (z : Set S) : Set S := {s | exists n, s in kpow z n}
```

Exercise 5.16

A fixed-point definition of $\text{neg} : \mathbb{C} \rightarrow \mathbb{C}$, together with `tailMap` and the Thue–Morse element `tElt = fix(tmOp)`.

Lean 4 source

```
def negMap : ApproximableMap C C :=
```

3.6 Lecture VI: Domain Equations

Example 6.1

The tree algebra $D^\sharp$ collects indefinite iteration of pairing into one domain, solving $D^\sharp \cong D + (D^\sharp \times D^\sharp)$. Tokens live in $\{1,2\}^* 0 \Delta$ as `List Bool` $\times$ α ; neighbourhoods are generated by leaf embeddings `embZero` and binary nodes `embPair`.

Lean 4 source

```
theorem dsharp_domain_equation :
```

Example 6.2

$B \cong B+B$ and $C \cong \{\{\Lambda\}\}+C+C$ as order-isos against the Lecture III sum. The same method yields $\text{Asys } n \cong \text{Asys } n^n + \text{Asys } n^n$, and eventually-periodic n -ary trees correspond to regular languages.

Lean 4 source

```
theorem B_domain_equation : B ~?= ? sum B B B_nonempty B_nonempty :=
```


Definition 6.3

A functor on a category (into itself) sends objects X to $T(X)$ and morphisms $f:X \rightarrow Y$ to $T(f):T(X) \rightarrow T(Y)$, preserving identities and composition. Lean packages a small `Category` and an `Endofunctor` on it; the running instance is `DomainObj` with morphisms `ApproximableMap` (laws from Theorem 2.5).

Lean 4 source

```
structure Endofunctor (Obj : Type u) [Category Obj] where
  obj : Obj -> Obj
  map : {X Y : Obj} -> Category.Hom X Y -> Category.Hom (obj X) (obj Y)
```

Definition 6.4

A T -algebra is a carrier E together with a structure map $\text{str} : T(E) \rightarrow E$. A homomorphism is a morphism of carriers making the obvious square commute: $\text{hom} \odot A.\text{str} = B.\text{str} \odot T.\text{map}$ hom .

Lean 4 source

```
structure TAlgebra (T : Endofunctor Obj) where
  carrier : Obj
  str : Category.Hom (T.obj carrier) carrier
```

Definition 6.5

A T -algebra is initial when there is a unique homomorphism from it into every other T -algebra. The structure `IsInitial` records the chosen homomorphism `desc` and its uniqueness `uniq`.

Lean 4 source

```
structure IsInitial (A : TAlgebra T) where
  desc : (B : TAlgebra T) -> AlgHom A B
  uniq : forall (B : TAlgebra T) (h : AlgHom A B), h = desc B
```

Proposition 6.6

any two initial T -algebras are uniquely isomorphic.

Lean 4 source

```
def initialIso (hA : IsInitial A) (hB : IsInitial B) : Iso A.carrier B.carrier where
```

Proposition 6.7

if $i:T(D) \rightarrow D$ is initial then so is $T(i)$, and i is an isomorphism (Lambek).

Lean 4 source

```
def lambek (A : TAlgebra T) (hA : IsInitial A) : Iso (T.obj A.carrier) A.carrier where
```

Definition 6.8

On domains and strict maps, T is continuous on maps when $\lambda f. T(f)$ is approximable as a map of strict function spaces. The identity endofunctor is continuous on maps.

Lean 4 source

```
def ContinuousOnMaps (T : Endofunctor DomainObj) : Prop :=
```

Theorem 6.9

if T is continuous on maps and $D \cong T(D)$, then a homomorphism $D \rightarrow E$ exists into any T -algebra with strict structure map.

Lean 4 source

```
theorem nonempty_algHom_of_continuousOnMaps
  (T : Endofunctor DomainObj) (hT : ContinuousOnMaps T)
  {D : DomainObj} (iso : Iso (T.obj D) D)
  (B : TAlgebra T) (hk : IsStrict B.str) :
```

Definition 6.10

The subsystem relation $D \triangleleft E$ means every D -neighbourhood is an E -neighbourhood and consistency is inherited: if $X, Y \in D$ and $X \cap Y \in E$ then $X \cap Y \in D$. The relation is a partial order on systems over a common token type.

Lean 4 source

```
structure Subsystem (D E : NeighborhoodSystem a) : Prop where
```

Proposition 6.11

the subsystems of E , ordered by $\triangleleft$, form a domain.

Lean 4 source

```
def subsystemReprIso (E : NeighborhoodSystem a) :
```

Proposition 6.12

$D \triangleleft E$ yields a projection pair $i: D \rightarrow E$, $j: E \rightarrow D$ with $j \circ i = I_D$ and $i \circ j \sqsubseteq I_E$.

Lean 4 source

```
def inj (h : D ? E) : ApproximableMap D E where
```

Definition 6.13

T is monotone on domains when $D \triangleleft E$ implies $T(D) \triangleleft T(E)$ and the 6.12 pair is sent to $T(i), T(j)$. It is continuous on domains when that action preserves directed unions of subsystems.

Lean 4 source

```
def MonotoneOnDomains (T : Endofunctor DomainObj.{w}) : Prop :=
```

Theorem 6.14

A functor continuous on maps and monotone and continuous on domains, together with a generating set Γ such that $\{\Gamma\} \triangleleft T(\{\Gamma\})$, has a solution $D \cong T(D)$ — the colimit of the tower $T^n(\{\Gamma\})$.

Lean 4 source

```
structure Setup where
  T : Endofunctor DomainObj.{w}
  hmaps : ContinuousOnMaps T
  hmono : MonotoneOnDomains T
  hcont : ContinuousOnDomains T
```

Lemma 6.15

a projection pair i, j with $j \circ i = I_D$ and $i \circ j \sqsubseteq I_E$ yields $D \trianglelefteq E$ (converse to 6.12).

Lean 4 source

```
theorem triangleleftteq_of_projectionPair (hji : j.comp i = idMap D)
  (hij : i.comp j <= idMap E) : D ? E :=
```

Theorem 6.16

an initial T -algebra D embeds in every other solution: $D \leq E$ whenever $E \cong T(E)$.

Lean 4 source

```
theorem trianglelefteq_of_isInitial
  (T : Endofunctor DomainObj.{w}) (hT : ContinuousOnMaps T)
  (Dalg : TAlgebra T) (hinit : IsInitial Dalg)
  (E : DomainObj.{w}) (isoE : Iso (T.obj E) E) :
```

Exercise 6.17

C is the initial algebra for $T(X) = \mathbb{N} + X + X$ among strict, $\emptyset$ -free domains. The n -ary generalization C_n is the corresponding initial algebra for n successors.

Lean 4 source

```
def CisInitial : IsInitial Calg where
```

Exercise 6.18

D^∞ is the initial algebra of $T(X) = D \times X$ in the strict category, solving $D^\infty \cong D \times D^\infty$.

Lean 4 source

```
def iterIsInitial (Dom : StrictDomainObj.{w}) : IsInitial (iterAlg Dom) where
```

Exercise 6.19

Uniform token-level sum and product over $\Delta \subseteq \{0,1\}^*$ ($\Lambda \in \Delta$, $\emptyset \notin \Delta$) are isomorphic to the Lecture III constructs.

Lean 4 source

```
def sumTokEquiv : (sumTok D_0 D_1 h_0 h_1).Element ?o (sum D_0 D_1 h_0 h_1).Element where
```

Exercise 6.20

$\lambda \Gamma. \text{tok}(T(\{\Gamma\}))$ is continuous on nonempty subsets of $\{0,1\}^*$, so a fixed point $\Gamma = \text{tok}(T(\{\Gamma\}))$ exists and Theorem 6.14 applies.

Lean 4 source

```
def mFun : FExpr -> Set Str -> Set Str
```

Exercise 6.21

The same fixed-point machinery with coalesced sum $\oplus$ and smash $\otimes$, and with n -ary generated functors.

Lean 4 source

```
def oplusTok (D_0 D_1 : NeighborhoodSystem Str)
```

Exercise 6.22

Each displayed equation is an instance of the 6.21/6.20 machinery, so each has a solution.

Lean 4 source

```
def Cnat : ScottSys where
```

Exercise 6.23

The initial solution of $\text{Exp} \cong \mathbb{N} \oplus ((\text{Exp} \times \text{Exp}) + (\text{Exp} \times \text{Exp}))$ as a GExpr algebra.

Lean 4 source

```
def Exp (N : ScottSys) (hN : ([ ] : Str) in N.sys.master) : ScottSys :=
```

Exercise 6.24

A coupled system $D \cong D+(D \times E)$, $E \cong D+E$ solved by a double token-set fixed point.

[Lean 4 source](#)

```
def gTok (p q : Set Str) : Set Str := insert ([ ] : Str) (embBit false p U embBit true q)
```

Exercise 6.25

For a projection pair, $j(y)$ is the directed sup of $\{x \mid i(x) \sqsubseteq y\}$ and $i(x)$ is the inf of $\{y \mid x \sqsubseteq j(y)\}$.

[Lean 4 source](#)

```
def lowerSet (P : ProjectionPair D E) (y : E.Element) : Set D.Element :=
```

Exercise 6.26

Lifting $D_{\perp}$ over $\{0,1\}^*$, with $D_{\perp} \oplus E_{\perp} \cong D+E$ and $D_{\perp} \otimes E_{\perp} \cong (D \times E)_{\perp}$.

[Lean 4 source](#)

```
def liftTok (D : NeighborhoodSystem Str) (_hD : forall X, D.mem X -> X.Nonempty) :
```

Exercise 6.27

The first five embeddings hold for all D, E ; $D \leq D \otimes E$ fails in general.

[Lean 4 source](#)

```
theorem not_triangleleftteq_otimes :
```

Exercise 6.28

Finite Cantor–Schröder–Bernstein: if $|D|, |E|$ are finite and $D \leq E \leq D$ then $D \cong^D E$.

[Lean 4 source](#)

```
theorem isomorphic_of_triangleleftteq_both [Finite D.Element] [Finite E.Element]
```

Exercise 6.29

Infinitary product and sum over an index family exist; infinite smash degenerates.

[Lean 4 source](#)

```
def iprod : NeighborhoodSystem (Sigma i, a i) where
```

3.7 Lecture VII: Computability in Effectively Given Domains

Definition 7.1

A *computable presentation* enumerates the neighbourhoods of D so that membership, intersection equality, and consistency are recursive. $X : \mathbb{N} \rightarrow \mathbf{Set}$ hits every neighbourhood ($\text{mem}_X, \text{surj}$); $X_n \cap X_m = X_k$ is recursively decidable, as is “ X_n and X_m are consistent”. A system is *effectively given* when some such presentation exists.

[Lean 4 source](#)

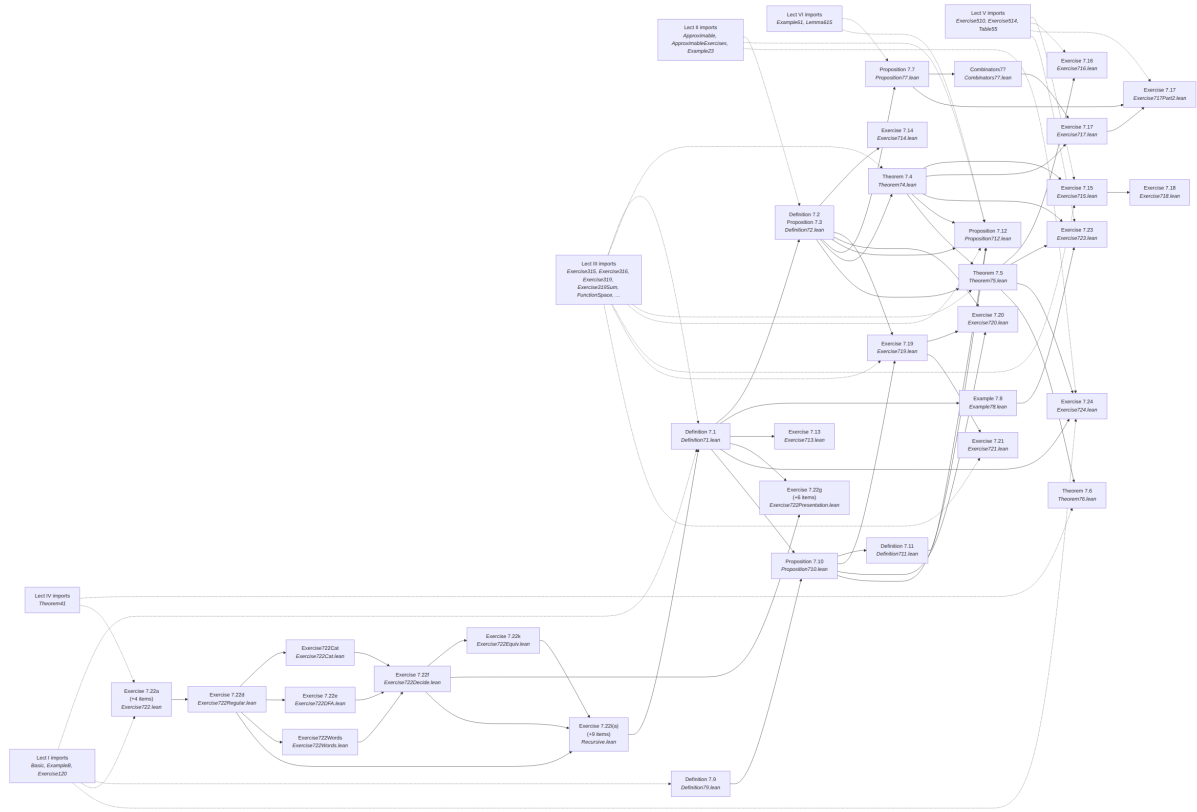


Figure 8: Lean module dependencies for Lecture VII: Computability in Effectively Given Domains.

```

structure ComputablePresentation (V : NeighborhoodSystem a) where
  X : Nat -> Set a
  mem_X : forall n, V.mem (X n)
  surj : forall {Y : Set a}, V.mem Y -> exists n, X n = Y
  interEq_computable : RecDecidable_3 (fun n m k => X n ∩ X m = X k)
  cons_computable : RecDecidable_2 (fun n m => exists k, X k ⊆ X n ∩ X m)

```

Definition 7.2

A map $f:D \rightarrow E$ between presented domains is *computable* when the index relation $X_n \cap Y_m$ is recursively enumerable. An element $y \in |E|$ is computable when $\{m \mid Y_m \in y\}$ is r.e.

Lean 4 source

```

def IsComputableMap {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
  (P : ComputablePresentation V) (Q : ComputablePresentation W) (f : ApproximableMap V W) :
  Prop :=
  REPred_2 (fun n m => f.rel (P.X n) (Q.X m))

```

Proposition 7.3

identity is computable; computable maps compose; a computable map sends computable elements to computable elements.

Lean 4 source

```

theorem idMap_isComputable (P : ComputablePresentation V) :

```

Theorem 7.4

if D_0, D_1 are effectively given then so are $D_0 \times D_1$ and $D_0 + D_1$, and the pairing, product, injection, and case maps are computable.

Lean 4 source

```
theorem prod_isEffectivelyGiven (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.IsEffectivelyGiven) :
```

Theorem 7.5

if D_0, D_1 are effectively given then so is $D_0 \rightarrow D_1$; `eval` and `curry` are computable; computable elements of the function space are exactly the computable maps.

Lean 4 source

```
def funPresentation (gN incl0 incl1 eq1 : Nat -> Nat)
```

Theorem 7.6

$\text{fix} : (D \rightarrow D) \rightarrow D$ is computable on every effectively given D .

Lean 4 source

```
theorem fixMap_isComputable
  (gN incl eq : Nat -> Nat)
```

Proposition 7.7

$D^\wedge$ is effectively given whenever D is, and the Example 6.1 combinators are computable.

Lean 4 source

```
def dsharpPresentation (hD : forall X, D.mem X -> X.Nonempty) :
```

Example 7.8

The powerset PN is effectively given: neighbourhoods are $\text{nbhd } n = \{k \mid n.\text{testBit } k = \text{false}\}$, closed under intersection via bitwise OR.

Lean 4 source

```
def PN : NeighborhoodSystem Nat where
```

Definition 7.9

The Smyth power domain starts from Scott's down-set $\downarrow X = \{Y \in D \mid Y \subseteq X\}$, which is Exercise 1.20's `upSet`. The preparation $D^\dagger = \{\downarrow X \mid X \in D\}$ is `powerSystem`, aliased `dagger`, and is isomorphic to D . Finite unions of down-sets form the family `PDmem`.

Lean 4 source

```
abbrev dagger : NeighborhoodSystem (Set a)
```

Proposition 7.10

PD is a neighbourhood system if D is, and it is effectively given if D is.

Lean 4 source

```
abbrev PowerDomain : NeighborhoodSystem (Set a) where
```

Definition 7.11

Finite-element joins $\{x_0, \dots, x_{n-1}\}$ in the power domain, with $\{x\}$ the singleton embedding.

Lean 4 source

```
def PDfinJoin (n : Nat) (xs : Fin n -> V.Element) : V.PowerDomain.Element :=
```

Proposition 7.12

The singleton map $D \rightarrow PD$ is approximable (and computable when D is effectively given). The embedding $D \sqsubseteq PD$ fails in general: **Vshape** is a counterexample.

Lean 4 source

```
def PDSingletonApproxMap : ApproximableMap V V.PowerDomain :=
```

Exercise 7.13

An effectively given domain is determined by a recursively decidable inclusion $INCL(n, m)$ on indices, with recursive meet and consistency.

Lean 4 source

```
structure InclStructure where
  INCL : Nat -> Nat -> Prop
  meetIdx : Nat -> Nat -> Nat
  topIdx : Nat
```

Exercise 7.14

A nonempty r.e. set is the range of a primitive recursive function; every computable element is a decreasing union of principals.

Lean 4 source

```
theorem computableElement_eq_decreasing_iUnion_principal {W : NeighborhoodSystem b}
```

Exercise 7.15

Smash, coalesced sum, and D^∞ are effectively given when the factors are, with the corresponding combinators computable.

Lean 4 source

```
theorem smash_isEffectivelyGivenS (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.IsEffectivelyGiven)
  :
```

Exercise 7.16

curry as a neighbourhood relation is recursively decidable, not merely r.e.

Lean 4 source

```
theorem curryComb_rel_recDecidable
```

Exercise 7.17

The remaining $D^\S$ combinators are computable; a unique strict $g : D^\S \rightarrow E$ exists for computable $u : D \rightarrow E$ and $v : E \times E \rightarrow E$.

Lean 4 source

```
def gMap (u : ApproximableMap D E) (v : ApproximableMap (prod E E) E)
```

Exercise 7.18

An *effective isomorphism* is a pair of mutually inverse computable maps. If D is effectively given then $D^\infty \cong (D^\infty)^\infty$ effectively.

Lean 4 source

```
structure EffectiveIso {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
```

Exercise 7.19

$D \mapsto PD$ is a functor: $PFmap\ f : PD \rightarrow PE$ is approximable, preserves identity and composition, and is computable when f is.

Lean 4 source

```
def PFmap (f : ApproximableMap V W) : ApproximableMap V.PowerDomain W.PowerDomain where
```

Exercise 7.20

The flattening combinator $union : P(PD) \rightarrow PD$ (Smyth monad multiplication) is approximable and computable.

Lean 4 source

```
def unionMap (V : NeighborhoodSystem a) :
```

Exercise 7.21

A non-trivial combinator of type $P(D \rightarrow E) \rightarrow (PD \rightarrow PE)$: pointwise application $papply$.

Lean 4 source

```
def ppapply (V : NeighborhoodSystem a) (W : NeighborhoodSystem b) :
```

Exercise 7.22a

Least-fixed-point family S over $\{0,1\}^*$, generated by $univ$, singletons, concatenation, and nonempty intersection.

Lean 4 source

```
inductive InS : Set (List Bool) -> Prop
```

Exercise 7.22b

S is a positive neighbourhood system on $\{0,1\}^*$.

Lean 4 source

```
def Ssys : NeighborhoodSystem (List Bool) where
```

Exercise 7.22c

Concatenation of filters is an associative semigroup operation on $|S|$, and embeds the free monoid.

Lean 4 source

```
def mulElem (x y : Ssys.Element) : Ssys.Element where
```

Exercise 7.22d

Regular-event syntax $SExpr$ (no union, complement, or star) with decidable membership.

Lean 4 source

```
inductive SExpr : Type
```

Exercise 7.22e

Every $SExpr$ language is accepted by a finite automaton.

Lean 4 source

```
def toNFA : (e : SExpr) -> NFA Bool (autState e)
```

Exercise 7.22f

Executable emptiness and consistency deciders for Def 7.1 (ii) on syntax.

Lean 4 source

```
def consistentB (a b : SExpr) : Bool := !decideEmptyB (.cap a b)
```

Exercise 7.22g

Gödel enumeration SsysX of the members of S .

Lean 4 source

```
def SsysX (n : Nat) : Set (List Bool) :=
```

Exercise 7.22h

Infinite-word filters `streamElem w`, with idempotence of `w-vec` when $\{w^n\}$ lies in S .

Lean 4 source

```
def streamElem (w : List Bool) : Ssys.Element where
```

Exercise 7.22i(a)

Primitive-recursive $\{0,1\}$ -digit validation on coded lists.

Lean 4 source

```
def isBinDigit (n : Nat) : Nat :=
```

Exercise 7.22i(b)1(a)

Coded $\{0,1\}$ AND.

Lean 4 source

```
def mulBit (a b : Nat) : Nat := a * b
```

Exercise 7.22i(b)1(b)

Fuel-bounded decode-ok character for Gödel codes of `SExpr`.

Lean 4 source

```
def decodeFuelOkChar : Nat -> Nat -> Nat
```

Exercise 7.22i(b)1(c)

Tag-dispatch equation linking the decode-ok body to the four `SExpr` constructors.

Lean 4 source

```
theorem decodeFuelOkCharBody_eq (prev : Nat -> Nat) (c : Nat) :
```

Exercise 7.22i(b)1(d)

List-of-bits decode succeeds iff every digit is binary.

Lean 4 source

```
theorem decodeListBool_isSome_iff (n : Nat) :
```

Exercise 7.22i(b)1(e)

The decode-ok character is 1 exactly when fuel-bounded decode returns `some`.

Lean 4 source

`theorem decodeFuel0kChar_eq_one_iff (fuel c : Nat) :`

Exercise 7.22i(b)1

`decodeFuel0kChar` is jointly primitive recursive in `fuel` and `code`.

Lean 4 source

`theorem primrec_decodeFuel0kChar : forall fuel, Nat.Primrec (fun c => decodeFuel0kChar fuel c)`

Exercise 7.22i(b)2

Coded-list length as a primitive-recursive character.

Lean 4 source

`def listLenChar (c : Nat) : Nat := foldCode listLenStp 0 0 c`

Exercise 7.22i(b)3

Coded-list equality as a primitive-recursive character.

Lean 4 source

`def listEqChar (c1 c2 : Nat) : Nat :=`

Exercise 7.22i(b)4

Append, take, and drop on list codes, all primitive recursive.

Lean 4 source

`def appendListCode (c1 c2 : Nat) : Nat :=`

Exercise 7.22i(b)5

Numeric `matchesB` and a state-card bound, both primitive recursive.

Lean 4 source

`def matchesBChar : Nat -> Nat -> Nat -> Nat`

Exercise 7.22i(b)6

Emptiness and cap-consistency characters by bounded index search.

Lean 4 source

`def decideNonemptyBChar (fuel : Nat) (c_e : Nat) : Nat :=`

Exercise 7.22i(b)7

Index-level activity and consistency characters on `SsysX`.

Lean 4 source

`def ssysActiveChar (n : Nat) : Nat :=`

Exercise 7.22i(b)8

Course-of-values tabulation making the fuel-recursive characters jointly primitive recursive.

Lean 4 source

`def fuelTable (bodyLookup : Nat -> Nat) (bound fuel : Nat) : Nat :=`

Exercise 7.22i(b)

Consistency on the `SsysX` enumeration is recursively decidable (Def 7.1 (ii)).

Lean 4 source

```
theorem Ssys_cons_computable : RecDecidable_2 (fun n m => exists k, SsysX k subseq SsysX n
  I SsysX m) :=
```

Exercise 7.22j

Ssys carries a consistency presentation: an enumeration with recursive consistency.

Lean 4 source

```
def SsysPres : ConsistencyPresentation Ssys where
```

Exercise 7.22k

Intersection equality on SsysX is recursively decidable, via language equivalence of regular events.

Lean 4 source

```
theorem Ssys_interEq_computable :
```

Exercise 7.22l

Scott's infinite-word equations hold when sigma-vec is the least fixed point of prefixing by σ .

Lean 4 source

```
def streamArrow (sigma : List Bool) : Ssys.Element := (prependMap sigma).fixElement
```

Exercise 7.23

On PN, the maps $\cap$, $\cup$, $+$, fun, and graph are computable; computable elements are exactly those whose bit-set is r.e.

Lean 4 source

```
theorem isComputableElement_iff_elemSet_re (x : PN.Element) :
```

Exercise 7.24

Γ is the set of finite and infinite sequences of naturals; L is the cone system of finite prefixes. L is effectively given, $|L|$ is in bijection with Γ , and B embeds into L by reading bits as 0,1. LUCID combinators lift along postcompose and pointwiseBin to computable maps of type $(L \rightarrow T) \rightarrow (L \rightarrow T)$ and $(L \rightarrow T) \times (L \rightarrow T) \rightarrow (L \rightarrow T)$.

Lean 4 source

```
abbrev L : NeighborhoodSystem Gamma :=
```

Lean 4 source

```
theorem toElement_bijective : Function.Bijective toElement :=
```

Lean 4 source

```
def postcompose {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem ?}
  (h : ApproximableMap V_1 V_2) : ApproximableMap (funSpace L V_1) (funSpace L V_2) :=
```


3.8 Lecture VIII: Retracts of the Universal Domain

Retractions, projections, and the universal domain U . Scott prints Theorem 8.8 as one unlettered three-sentence theorem; this inventory labels those sentences 8.8(a)/(b)/(c) for Lean names only.

Definition 8.1

A retraction is an approximable self-map with $a \setminus \text{circa} = a$.

Lean 4 source

```
def IsRetraction (a : ApproximableMap E E) : Prop
```

Proposition 8.2

A subsystem $D \setminus \text{triangleleft} E$ induces the retraction $a = i \setminus \text{circ} j$, with $\|D\| \cong \text{Fix}(a)$.

Lean 4 source

```
theorem isRetraction_retractionOfSubsystem (h : D ? E) :
```

Definition 8.3

A projection is a retraction $a \setminus \text{sqsubseteq} I$. It is finitary when $\text{Fix}(a)$ is order-isomorphic to a domain.

Lean 4 source

```
def IsProjection (a : ApproximableMap E E) : Prop
```

Example 8.4(a)

On non-trivial D , $O = \{\{0\}, \{0, 1\}\}$ arises as $a(x) = \text{fade}(\text{check}(x), u)$ for $u \setminus \text{neg} \setminus \text{bot}$; $\text{range} \cong O$.

Lean 4 source

```
def 0 : NeighborhoodSystem (Fin 2)
```

Example 8.4(b)

Same combinators: smash on $D \setminus \text{times} E$ ($\text{range} \cong D \setminus \text{otimes} E$) and strict on $D \setminus \text{to} E$.

Lean 4 source

```
theorem smashRetraction_eq_retractionOfSubsystem :
```

Theorem 8.5

A self-map is a finitary projection iff $a(x) = \{Y \setminus \text{in} E \setminus \text{mid} \setminus \text{exists} X \setminus \text{inx}, X \setminus \text{subseq} Y \setminus \text{land} X a X\}$.

Lean 4 source

```
theorem finitaryProjection_iff_formula (a : ApproximableMap E E) :
```

Theorem 8.6(a)

$\text{sub } f$ is the retraction of $\{Y \setminus \text{mid} Y f Y\}$; its range is the finitary projections.

Lean 4 source

```
def sub (f : ApproximableMap E E) : ApproximableMap E E
```

Theorem 8.6(b)(i)

$f \setminus \text{mapsto} \setminus \text{mathrmsub}(f)$ is itself a projection on $E \setminus \text{to} E$.

Lean 4 source

`theorem isProjection_subApprox`

Theorem 8.6(b)(ii)

$\text{Fix}(\text{subApprox})$ is isomorphic to a domain, so sub is finitary.

[Lean 4 source](#)

`theorem isFinitary_subApprox`

Theorem 8.6(c)

If E is effectively given, sub is computable.

[Lean 4 source](#)

`theorem subApprox_isComputable`

Definition 8.7

Non-empty finite unions of rational intervals $[r, s] \setminus \text{subsetq}[0, 1)$. Membership is presentability plus $X \setminus \text{neq} \setminus \text{emptyset}$ and $X \setminus \text{subsetq}[0, 1)$. Intersection is pairwise endpoint combination; there are no minimal neighbourhoods.

[Lean 4 source](#)

`def U : NeighborhoodSystem ?`

Theorem 8.8, first sentence (editorial label 8.8(a))

Every countable neighbourhood system embeds in U : some isomorphic copy is a subsystem of U .

[Lean 4 source](#)

`theorem theorem_8_8`

Theorem 8.8(b)

Effective refinement: if D is effectively given, the witnessing projection pair may be taken computable.

[Lean 4 source](#)

`theorem theorem_8_8_b`

Theorem 8.8(b)(i)

Choice-free Gödel numbering of $\mathbb{Z}$ and $\mathbb{Q}$.

[Lean 4 source](#)

`def encodeRat`

Theorem 8.8(b)(ii)

List-coded interval unions, with primitive-recursive combine and difference.

[Lean 4 source](#)

`def combineCode`

Theorem 8.8(b)(iii)

A computable presentation of U .

[Lean 4 source](#)

`def UComputablePresentation`

Theorem 8.8(b)(iv)

A deterministic midpoint split of each U -code, replacing the existential no-minimal witness.

[Lean 4 source](#)

`def splitULeft`

Theorem 8.8(b)(v)

Satisfiability of a finite Boolean atom-constraint on D is recursively decidable.

[Lean 4 source](#)

`theorem DAtom_recDecidable`

Theorem 8.8(b)(vi)

The Y_n chain, parametrized by an abstract split satisfying `SplitSpec`.

[Lean 4 source](#)

`def SplitSpec`

Theorem 8.8(b)(vii)

The code-native projection pair $D'' \hookrightarrow U$ is computable.

[Lean 4 source](#)

`theorem DprimeUCode_projectionPair_isComputable`

Theorem 8.8(b)(vii)(1)

Code-level atom invariant: every `atomUCode` is a U -neighbourhood, and distinct non-junk bit-sources are disjoint.

[Lean 4 source](#)

`theorem atomUCode_disjoint`

Theorem 8.8(b)(vii)(2)

Scott's Y_n as a primitive-recursive union of 2^n atoms.

[Lean 4 source](#)

`noncomputable def YseqCode`

Theorem 8.8(b)(vii)(3)

The subsystem D'' with $X\ n = UX(YseqCode\ n)$ is isomorphic to D and a subsystem of U .

[Lean 4 source](#)

`noncomputable def DprimeUCode`

Theorem 8.8(b)(vii)(4)

Inclusion against U 's presentation makes D'' 's injection and projection computable.

[Lean 4 source](#)

`theorem DprimeUCode_inj_isComputableMap`

Theorem 8.8(b)(viii)

Assembles the effective refinement: an isomorphic copy $D' \triangleleft U$ with a computable projection pair.

[Lean 4 source](#)

```
theorem theorem_8_8_b
```

Theorem 8.8(c)

Finitary-projection correspondence: a computable finitary projection of U yields an effectively given domain.

[Lean 4 source](#)

```
theorem theorem_8_8_c {a : ApproximableMap U U} (_hfin : IsFinitaryProjection a)
```

Theorem 8.8(c)(i)

The diagonal $X_n a X_n$ is r.e. when a is computable.

[Lean 4 source](#)

```
def DiagFixed
```

Theorem 8.8(c)(ii)

A primitive-recursive fold whose output index always satisfies `DiagFixed`.

[Lean 4 source](#)

```
def myFoldCode
```

Theorem 8.8(c)(iii)

The induced enumeration covers `fixedNbhd a` exactly.

[Lean 4 source](#)

```
def D_X
```

Theorem 8.8(c)(iv)

D_X 's intersection-equality and consistency relations are recursively decidable.

[Lean 4 source](#)

```
theorem D_X_interEq_computable
```

Theorem 8.8(c)(v)

List-append is a primitive-recursive intersection index for D_X .

[Lean 4 source](#)

```
theorem D_X_inter_spec
```

Theorem 8.8(c)(vi)

Packages (i)–(v) into a computable presentation of `fixedNbhd a`.

[Lean 4 source](#)

```
theorem fixedNbhd_isEffectivelyGiven
```

Definition 8.9

Computable pairs $i_+, j_+ / i_\times, j_\times / i_\rightarrow, j_\rightarrow$ for $U + U$, $U \setminus \text{times} U$, $U \setminus \text{to} U$, plus combinators $a + b$, $a \setminus \text{times} b$, $a \setminus \text{to} b$.

Lean 4 source

```
noncomputable def iPlus : ApproximableMap (sum U U U_mem_nonempty U_mem_nonempty) U
```

Proposition 8.10(a)

If a, b are projections of U , so are $a + b$, $a \setminus \text{times} b$, and $a \setminus \text{to} b$.

Lean 4 source

```
theorem isProjection_combinators {a b : ApproximableMap U U}
```

Proposition 8.10(b)

If a, b are finitary, so are the three combinators, with $D_{a*b} \cong D_a * D_b$.

Lean 4 source

```
theorem finitaryProjection_combinators {a b : ApproximableMap U U}
```

Exercise 8.11

On nested $[0, r)$, the map $r a s$ iff $r < s$ or $r = s = 1$ is a non-finitary projection; $\text{Fix}(a) \setminus \text{left} \text{right} \text{arrow} [0, 1]$.

Lean 4 source

```
def a : ApproximableMap R R
```

Exercise 8.12

Finite unions of $2^k \setminus \text{mathbb{N}} + \ell$ form $V \setminus \text{cong} U$ effectively, a second presentation of U .

Lean 4 source

```
def V : NeighborhoodSystem Nat
```

Exercise 8.12(a)

Defines V via bitmasks $\text{levelSet } k m = \{n \setminus \text{mid } m.\text{testBit}(n \setminus b \text{ mod } 2^k)\}$.

Lean 4 source

```
def V : NeighborhoodSystem Nat
```

Exercise 8.12(b)

A computable presentation of V , including primitive-recursive bit arithmetic and upsampling.

Lean 4 source

```
def VComputablePresentation : ComputablePresentation V
```

Exercise 8.12(c)

Two countable atomless systems, each able to split against the other, are isomorphic.

Lean 4 source

```
theorem isomorphic_812c : D_0 ~ = ? D_1
```

Exercise 8.12(c)(i)

U and V are closed under set-difference (or empty).

[Lean 4 source](#)

```
theorem U_diff_mem
```

Exercise 8.12(c)(ii)

Abstract atomlessness and difference-closure; finite Boolean atoms are then mem-or-empty.

[Lean 4 source](#)

```
def NeighborhoodSystem.NoMinimal
```

Exercise 8.12(c)(iii)

The one-step split, now for an arbitrary atomless target.

[Lean 4 source](#)

```
theorem exists_split'
```

Exercise 8.12(c)(iv)

Interleaved two-sided atoms with matched emptiness on both sides.

[Lean 4 source](#)

```
noncomputable def atomPair
```

Exercise 8.12(c)(v)

Disagreeing sign sequences yield disjoint atoms on both sides.

[Lean 4 source](#)

```
theorem atomPair_disjoint
```

Exercise 8.12(c)(v)(1)

One-step intersect/subtract-and-split, with disjointness at opposite signs.

[Lean 4 source](#)

```
def xyStep
```

Exercise 8.12(c)(v)(2)

Named X - and Y -substeps as instances of $xyStep$.

[Lean 4 source](#)

```
noncomputable def xStep
```

Exercise 8.12(c)(v)(3)

The recursive step of $atomPair$ is $yStep \setminus circxStep$.

[Lean 4 source](#)

```
theorem atomPair_succ_eq
```

Exercise 8.12(c)(v)(4)

Atoms shrink with depth on both sides.

[Lean 4 source](#)

`theorem atomPairfst_subset`

Exercise 8.12(c)(v)(5)

Headline two-sided disjointness, by induction on depth.

[Lean 4 source](#)

`theorem atomPair_disjoint`

Exercise 8.12(c)(vi)

Each enumeration index is a finite union of matched atoms, so subset and intersection transfer both ways.

[Lean 4 source](#)

`theorem X_subset_iff_XPseq_subset`

Exercise 8.12(c)(vi)(1)

One-sided $Yseq$ closed forms over an abstract atomless E .

[Lean 4 source](#)

`theorem atomE_eq_genAtom`

Exercise 8.12(c)(vi)(2)

Finite-constraint transfer lemmas, now over that abstract E .

[Lean 4 source](#)

`theorem transfer_subset_iffE`

Exercise 8.12(c)(vi)(3)

Those unions are empty or genuine neighbourhoods.

[Lean 4 source](#)

`theorem YseqE_empty_or_mem`

Exercise 8.12(c)(vi)(4)

Recovers X_n/Y_n as unions of opposite-side half-step pieces.

[Lean 4 source](#)

`noncomputable def XPseq`

Exercise 8.12(c)(vi)(4)(a)

The X -side half-step union over depth- n histories.

[Lean 4 source](#)

`theorem xStep_snd_eq_inter_XPseq`

Exercise 8.12(c)(vi)(4)(b)

The Y -side half-step union, with an extra free X -bit.

[Lean 4 source](#)

`noncomputable def YPseq`

Exercise 8.12(c)(vi)(4)(c)

Transports $\text{SplitSpec}'$ across the X -substep at an arbitrary bit.

[Lean 4 source](#)

```
theorem xStep_spec_bit
```

Exercise 8.12(c)(vi)(4)(d)

The Y -side I-formula: the $+$ -branch equals intersection with $\text{YPseq } n$.

[Lean 4 source](#)

```
theorem yStep_fst_eq_inter_YPseq
```

Exercise 8.12(c)(vi)(5)

Both sides of atomPair are genAtom over interleaved families, so transfer applies.

[Lean 4 source](#)

```
theorem atomPair_fst_eq_genAtom
```

Exercise 8.12(c)(vi)(5)(a)

Two-branch closed forms for $x\text{Step}/y\text{Step}$ at either sign.

[Lean 4 source](#)

```
theorem xStep_snd_succ_eq
```

Exercise 8.12(c)(vi)(5)(b)

Interleaved families and the identification with genAtom .

[Lean 4 source](#)

```
noncomputable def combinedX
```

Exercise 8.12(c)(vi)(5)(c)

Matching emptiness of the two interleaved genAtoms .

[Lean 4 source](#)

```
theorem hcore
```

Exercise 8.12(c)(vi)(5)(c)(1)

De-interleaving a bit sequence and re-interleaving recovers it.

[Lean 4 source](#)

```
def deinterleave?
```

Exercise 8.12(c)(vi)(5)(c)(2)

Even-depth matching emptiness.

[Lean 4 source](#)

```
theorem hcore_even
```

Exercise 8.12(c)(vi)(5)(c)(3)

Odd-depth matching emptiness.

[Lean 4 source](#)

`theorem hcore_odd`

Exercise 8.12(c)(vi)(5)(c)(4)

Assembles even and odd cases into hcore.

[Lean 4 source](#)

`theorem hcore`

Exercise 8.12(c)(vi)(5)(d)

Same-parity transfer: $X_i \setminus \text{subseq} X_j$ iff $\text{XPseq } i \setminus \text{subseq} \setminus \text{mathrm{XPseq } } j$, and the intersection analogues.

[Lean 4 source](#)

`theorem X_subset_iff_XPseq_subset`

Exercise 8.12(c)(vi)(6)

The swapped-sides row is absorbed here: both half-step unions are genuine neighbourhoods.

[Lean 4 source](#)

`theorem XPseq_mem`

Exercise 8.12(c)(vi)(7)

Enumerated sets and their atom-images are never empty, hence always genuine neighbourhoods.

[Lean 4 source](#)

`theorem NeighborhoodSystem.NoMinimal.mem_ne_empty`

Exercise 8.12(c)(vii)

Assembles the two filter maps into $D_0 \setminus \text{cong} D_1$.

[Lean 4 source](#)

`theorem isomorphic_812c`

Exercise 8.12(c)(vii)(1)

Adds covering and master-at-zero hypotheses for the two enumerations.

[Lean 4 source](#)

`variable (hXcover : forall S, D_0.mem S <=> exists n, S = X n)`

Exercise 8.12(c)(vii)(2)

Zero-depth half-step unions recover the two masters.

[Lean 4 source](#)

`theorem XPseq_zero`

Exercise 8.12(c)(vii)(3)

Cross-parity order and an intersection-index lemma.

[Lean 4 source](#)

`theorem X_subset_YPseq_iff_XPseq_subset_Y`

Exercise 8.12(c)(vii)(4)

Pushforward filter along XP_{seq} .

[Lean 4 source](#)

```
def toD1
```

Exercise 8.12(c)(vii)(5)

Pullback filter along YP_{seq} .

[Lean 4 source](#)

```
def toD0
```

Exercise 8.12(c)(vii)(6)

The two maps are mutually inverse and order-reflecting.

[Lean 4 source](#)

```
noncomputable def domainIso812c
```

Exercise 8.12(d)

Effective refinement of (c): back-and-forth over computable splits, instantiated on U and V .

[Lean 4 source](#)

```
theorem effectivelyIsomorphic_812d : EffectivelyIsomorphic P_0 P_1
```

Exercise 8.12(d)(1)

Generalizes `atomPair` over abstract splits satisfying `SplitSpec'`.

[Lean 4 source](#)

```
noncomputable def atomPairG
```

Exercise 8.12(d)(2)

What it means for a split to be computable relative to two presentations.

[Lean 4 source](#)

```
structure IsComputableSplit
```

Exercise 8.12(d)(3)

A code-level two-sided atom machine, tracking indices and a junk flag.

[Lean 4 source](#)

```
noncomputable def atomPairCodeState
```

Exercise 8.12(d)(3)(a)

A computable difference-index, the missing companion of Definition 7.1's intersection index.

[Lean 4 source](#)

```
structure IsComputableDiff
```

Exercise 8.12(d)(3)(b)

The X -substep as a primitive-recursive state transition.

[Lean 4 source](#)

`noncomputable def xSubStep`

Exercise 8.12(d)(3)(c)

The Y -substep, composed with (b) into a full depth step.

[Lean 4 source](#)

`noncomputable def atomPairStep`

Exercise 8.12(d)(3)(d)

Non-junk packed indices recover `atomPairG`.

[Lean 4 source](#)

`theorem atomPairCodeState_correct`

Exercise 8.12(d)(3)(e)

Junk persists; non-junk indices are genuine neighbourhoods.

[Lean 4 source](#)

`theorem atomPairJunk_mono`

Exercise 8.12(d)(3)(f)

Disagreeing non-junk bit-sources remain disjoint.

[Lean 4 source](#)

`theorem atomPairCodeState_disjoint`

Exercise 8.12(d)(4)

Folds the 4^n half-step atoms into primitive-recursive union codes.

[Lean 4 source](#)

`noncomputable def XPseqCode`

Exercise 8.12(d)(4)(a)

A computable union-index, the missing companion of Definition 7.1 for $\cup$.

[Lean 4 source](#)

`structure IsComputableUnion`

Exercise 8.12(d)(4)(b)

A consistent union of two neighbourhoods is again a neighbourhood.

[Lean 4 source](#)

`theorem NeighborhoodSystem.mem_union_of_mem`

Exercise 8.12(d)(4)(c)

The X -side fold and its membership characterization.

[Lean 4 source](#)

`theorem mem_XPseqCode_iff_unconditional`

Exercise 8.12(d)(4)(c)(i): one-step 4-way classical reunion One-step four-way reunion of atomPairG on the D_0 side.

[Lean 4 source](#)

```
theorem atomPairGfst_union_step
```

Exercise 8.12(d)(4)(c)(ii): classical covering induction Covering induction: depth- n atoms exhaust the D_0 master.

[Lean 4 source](#)

```
theorem atomPairG_master_covered
```

Exercise 8.12(d)(4)(c)(iii): encode sign-histories as deltaPair-matching bit-sources Sign histories as base-4 bit-sources.

[Lean 4 source](#)

```
def deltaPair
```

Exercise 8.12(d)(4)(c)(iv): non-trivial intersection with $P_0.X_n$, still classical Some atom at depth n meets $P_0.X_n$ non-trivially.

[Lean 4 source](#)

```
theorem exists_atomPairG_deltaPair_inter_Xn_ne_empty
```

Exercise 8.12(d)(4)(c)(v): the converse-biconditional — (c)’s originally-named gap, now itself a concrete induction The fold’s found-flag matches existence of a non-junk half-step.

[Lean 4 source](#)

```
theorem XFold_found_iff
```

Exercise 8.12(d)(4)(c)(vi): assemble into the unconditional “found” fact Every $\text{XPseqCode } n$ is a genuine D_1 -neighbourhood.

[Lean 4 source](#)

```
theorem XPseqCode_mem_unconditional
```

Exercise 8.12(d)(4)(d)

The Y -side fold, with an extra Boolean layer.

[Lean 4 source](#)

```
noncomputable def YPseqCode
```

Exercise 8.12(d)(4)(d)(i): one-step 4-way classical reunion, D_1 -side One-step four-way reunion on the D_1 side.

[Lean 4 source](#)

`theorem atomPairG_snd_union_step`

Exercise 8.12(d)(4)(d)(ii): classical covering induction, D_1 -side Covering induction on the D_1 master.

[Lean 4 source](#)

`theorem atomPairG_master_covered_snd`

Exercise 8.12(d)(4)(d)(iii): encode sign-histories as deltaPair-matching bit-sources, D_1 -side The same deltaPair coding on the Y -side.

[Lean 4 source](#)

`theorem deltaPair_encodeDeltaPair`

Exercise 8.12(d)(4)(d)(iv): non-trivial intersection with $P_1.X\ n$, still classical Some atom at depth n meets $P_1.X\ n$ non-trivially.

[Lean 4 source](#)

`theorem exists_atomPairG_deltaPair_inter_Yn_ne_empty`

Exercise 8.12(d)(4)(d)(v): the converse-biconditional, D_1 -side The Y -fold found-flag, doubled over the extra bit.

[Lean 4 source](#)

`theorem YPseqCode_four_pow_found`

Exercise 8.12(d)(4)(d)(vi): assemble into the unconditional “found” fact, doubled over bx Every YPseqCode n is a genuine D_0 -neighbourhood.

[Lean 4 source](#)

`theorem YPseqCode_mem_unconditional`

Exercise 8.12(d)(5)

A generalized order-iso built on the code-level atoms, then shown computable.

[Lean 4 source](#)

`noncomputable def domainIsoCode812d`

Exercise 8.12(d)(5)(a): zero/master facts for XPseqCode/YPseqCode

Zero-depth codes recover the two masters.

[Lean 4 source](#)

`theorem XPseqCode_zero`

Exercise 8.12(d)(5)(b): the interleaving/order layer, generalized
Same-family subset and intersection transfer along XPseqCode/YPseqCode.

[Lean 4 source](#)

`theorem X_subset_iff_XPseqCode_subset`

Exercise 8.12(d)(5)(b)(iv): the headline transfer theorems The four headline transfer theorems.

[Lean 4 source](#)

`theorem X_inter_eq_iff_XPseqCode_inter_eq`

Exercise 8.12(d)(5)(c): cross-family order and equality facts
Cross-family order and equality.

[Lean 4 source](#)

`theorem X_subset_YPseqCode_iff_XPseqCode_subset_Y`

Exercise 8.12(d)(5)(c)(i): the cross-parity order fact, even/odd mix Even/odd mixed inclusion.

[Lean 4 source](#)

`theorem X_subset_YPseqCode_iff_XPseqCode_subset_Y`

Exercise 8.12(d)(5)(c)(ii): the cross-parity order fact, odd/even mix Odd/even mixed inclusion.

[Lean 4 source](#)

`theorem YPseqCode_subset_X_iff_Y_subset_XPseqCode`

Exercise 8.12(d)(5)(c)(iii): the cross-parity embed_eq_iff analogue Cross-parity equality of codes.

[Lean 4 source](#)

`theorem XPseqCode_eq_Y_iff_X_eq_YPseqCode`

Exercise 8.12(d)(5)(d): toD1Code/toD0Code, the generalized elementwise maps
Elementwise maps along the two code sequences.

[Lean 4 source](#)

`def toD1Code`

Exercise 8.12(d)(5)(d)(i): toD1Code's up_mem helper Upward closure for toD1Code.

[Lean 4 source](#)

`theorem toD1Code_up_mem`

Exercise 8.12(d)(5)(d)(ii): toD1Code, full assembly Assembles toD1Code as a D_1 -element.

[Lean 4 source](#)

`def toD1Code`

Exercise 8.12(d)(5)(d)(iii): toD0Code's up_mem helper Upward closure for toD0Code.

[Lean 4 source](#)

`theorem toD0Code_up_mem`

Exercise 8.12(d)(5)(d)(iv): toD0Code, full assembly Assembles toD0Code as a D_0 -element.

[Lean 4 source](#)

`def toD0Code`

Exercise 8.12(d)(5)(e): domainIsoCode812d, the generalized order isomorphism

The two maps form a domain isomorphism.

[Lean 4 source](#)

`theorem isomorphic_812d`

Exercise 8.12(d)(5)(e)(i): X_eq_iff_XPseqCode_eq, the same-family embed_eq_iff companion Same-family equality of codes.

[Lean 4 source](#)

`theorem X_eq_iff_XPseqCode_eq`

Exercise 8.12(d)(5)(e)(ii): the left_inv content, toD0Code (toD1Code x) = x Left inverse: $\text{toD0Code} \circ \text{toD1Code} = \text{id}$.

[Lean 4 source](#)

`theorem toD0Code_toD1Code`

Exercise 8.12(d)(5)(e)(iii): the right_inv content, toD1Code (toD0Code y) = y Right inverse: $\text{toD1Code} \circ \text{toD0Code} = \text{id}$.

[Lean 4 source](#)

`theorem toD1Code_toD0Code`

Exercise 8.12(d)(5)(e)(iv): `domainIsoCode812d/isomorphic_812d`, **full assembly** Packages the order-iso.

Lean 4 source

```
noncomputable def domainIsoCode812d
```

Exercise 8.12(d)(5)(f): computability — the exercise’s literal target

Both directions of the iso are computable maps.

Lean 4 source

```
theorem domainIsoCode812d_isComputableMap
```

Exercise 8.12(d)(5)(f)(i): `toD1Code_rel_iff` Unfolds `offIso` to an existential inclusion test.

Lean 4 source

```
theorem toD1Code_rel_iff
```

Exercise 8.12(d)(5)(f)(ii): `domainIsoCode812d_isComputableMap` That test is r.e., so the forward map is computable.

Lean 4 source

```
theorem domainIsoCode812d_isComputableMap
```

Exercise 8.12(d)(5)(f)(iii): `toD0Code_rel_iff` The inverse relation, via `YPseqCode`.

Lean 4 source

```
theorem toD0Code_rel_iff
```

Exercise 8.12(d)(5)(f)(iv): `domainIsoCode812d_symm_isComputableMap` The inverse map is computable.

Lean 4 source

```
theorem domainIsoCode812d_symm_isComputableMap
```

Exercise 8.12(d)(6)

Assembles an effective isomorphism of presentations.

Lean 4 source

```
theorem effectivelyIsomorphic_812d
```

Exercise 8.12(d)(6)(a): `invMap_comp_toMap`, **the left_inv content** Approximable-map left inverse of `offIso`.

Lean 4 source

```
theorem invMap_comp_toMap
```

Exercise 8.12(d)(6)(b): `toMap_comp_invMap`, the `right_inv` content Approximable-map right inverse.

[Lean 4 source](#)

```
theorem toMap_comp_invMap
```

Exercise 8.12(d)(6)(c): `effectiveIso812d/effectivelyIsomorphic_812d`, final assembly Packages `EffectiveIso`.

[Lean 4 source](#)

```
noncomputable def effectiveIso812d
```

Exercise 8.12(e)

A computable split of U against V , from a canonical bisection of V .

[Lean 4 source](#)

```
theorem isComputableSplit_812e :
```

Exercise 8.12(e)(a): the split’s contract, as Lean declarations

The contract of a computable bisection, and the two index selectors it induces.

[Lean 4 source](#)

```
structure ComputableBisection
```

Exercise 8.12(e)(b): `SplitV.lean` — a computable canonical bisection for V

A primitive-recursive bisection of each V -code into complementary residue classes.

[Lean 4 source](#)

```
theorem splitV_disjoint
```

Exercise 8.12(e)(b)(i): `myFirstBit` — the least-set-bit search combinator Least set-bit search.

[Lean 4 source](#)

```
theorem primrec_myFirstBit
```

Exercise 8.12(e)(b)(ii): a computable “clear one bit” / `xor-with-power-of-2` primitive Clear-one-bit / xor with a power of two.

[Lean 4 source](#)

```
theorem myClearBit_eq_xor
```

Exercise 8.12(e)(b)(iii): `splitVLeft/splitVRight` definitions and `Nat.Primrec` Left-/right split codes and their primitivity.

[Lean 4 source](#)

```
def splitVLeft
```

Exercise 8.12(e)(b)(iv): correctness — `VX_splitVLeft/VX_splitVRight`, `splitV_disjoint`, `splitV_union` The two pieces are complementary and exhaust the original code.

[Lean 4 source](#)

```
theorem splitV_union
```

Exercise 8.12(e)(c): generic decider+bisection $\rightarrow$ IsComputableSplit construction

Any computable bisection, with the emptiness deciders, yields a computable split.

[Lean 4 source](#)

```
noncomputable def isComputableSplit_ofBisection
```

Exercise 8.12(e)(c)(i): decider congruence and `posIdxFromBisection/negIdxFromBisection` well-definedness The deciders and selectors are well-defined on equal presentations.

[Lean 4 source](#)

```
theorem posIdxFromBisection_congr
```

Exercise 8.12(e)(c)(ii): `splitFromBisection` and `isComputableSplit_ofBisection` The set-level split and its `IsComputableSplit` witness.

[Lean 4 source](#)

```
noncomputable def splitFromBisection
```

Exercise 8.12(e)(d): instantiate for $U \leftrightarrow V$ — `splitX812e`, the exercise's literal target
Instantiates the generic split at U against V .

[Lean 4 source](#)

```
noncomputable def splitX812e
```

Exercise 8.12(e)(d)(i): minimal-level canonicalization for V 's codes Minimal-level canonicalization of a V -code.

[Lean 4 source](#)

```
def minLevel
```

Exercise 8.12(e)(d)(ii): redefine `splitVLeft/splitVRight` via the minimal presentation; `left_congr/right_congr` The bisection, rewritten on the minimal presentation, is congruence-invariant.

[Lean 4 source](#)

```
theorem splitVLeft_congr
```

Exercise 8.12(e)(d)(iii): package as `ComputableBisection`; instantiate for $U \leftrightarrow V$ Packages V 's bisection and the resulting $U \setminus to V$ split.

Lean 4 source

```
def B812e
```

Exercise 8.12(f)

The reverse split, using U 's existing midpoint bisection.

Lean 4 source

```
theorem hySplit812f : SplitSpec' U splitX812f
```

Exercise 8.12(g)

Instantiate (d) at the concrete U and V from (e)/(f).

Lean 4 source

```
noncomputable def effectiveIso812_UV : EffectiveIso UComputablePresentation
  VComputablePresentation
```

Exercise 8.12(g)(1)

Master non-emptiness and master-at-zero for both presentations.

Lean 4 source

```
theorem U_master_nonempty
```

Exercise 8.12(g)(1)(a)

U -side master facts.

Lean 4 source

```
theorem UX_zero
```

Exercise 8.12(g)(1)(b)

V -side master facts.

Lean 4 source

```
theorem VX_zero
```

Exercise 8.12(g)(2)

Computable union indices for both presentations.

Lean 4 source

```
def U_isComputableUnion
```

Exercise 8.12(g)(3)

Both concrete splits satisfy `SplitSpec'` (empty output when the probe is empty).

Lean 4 source

```
theorem hxSplit812e
```

Exercise 8.12(g)(4)

Feeds every U/V witness into `effectiveIso812d`.

Lean 4 source

```
noncomputable def effectiveIso812_UV
```

Exercise 8.13

Proper filters of the free Boolean algebra on $\aleph_0$ generators, as Lindenbaum algebra and as Cantor-space clopens.

Lean 4 source

```
theorem exercise813a : U ~ = ? V
```

Exercise 8.13(a)

V is that filter domain: non-empty sets generated by generator $i = \{n \mid n.\text{testBit } i\}$.

Lean 4 source

```
theorem V_mem_iff_generatedBy {X : Set Nat} :
```

Exercise 8.13(b)

Semantic quotient of propositional formulas realizes the same algebra.

Lean 4 source

```
def Lindenbaum : Type
```

Exercise 8.13(c)

Filters correspond (order-reversingly) to closed sets; opens are reached via ideals of clopens.

Lean 4 source

```
def genPoint (i : Nat) : Set (Nat -> Bool)
```

Exercise 8.13(c1)

Cantor-space clopens are exactly GeneratedBy genPoint.

Lean 4 source

```
theorem isClopen_genPoint (i : Nat) : IsClopen (genPoint i)
```

Exercise 8.13(c2)

The $\mathbb{N}$ and $\mathbb{N} \rightarrow \text{Bool}$ presentations of the free algebra are order-isomorphic.

Lean 4 source

```
theorem Lindenbaum.range_toSet' :
```

Exercise 8.13(c3)

In a Boolean algebra of sets, complements swap proper ideals with proper filters.

Lean 4 source

```
def dualFilter
```

Exercise 8.13(c4)

A proper open yields an ideal of clopens, hence (via (c3), (c2)) an element of V .

Lean 4 source

```
def idealOf
```

Exercise 8.14

A closure operator is an inflationary retraction; its fixed-point set is always finitary.

Lean 4 source

```
theorem isFinitary_of_isClosureOperator (h : IsClosureOperator a) : IsFinitary a
```

Exercise 8.15

If D is effectively given, so is $\{X \setminus \text{mid} X \setminus \text{triangleleft} D\}$; on U , computable elements are the effectively presented domains.

Lean 4 source

```
theorem subsystem_isEffectivelyGiven_of_isEffectivelyGiven (hD : D.IsEffectivelyGiven) :
```

Exercise 8.16

Finitary projections satisfy $a \setminus \text{sqsubseteq} b$ iff $D_a \setminus \text{triangleleft} D_b$; computable a makes D_a effectively given.

Lean 4 source

```
theorem isFinitaryProjection_le_iff_fixedNbhd_subsystem {a b : ApproximableMap E E}
```

Exercise 8.17

The Definition 8.9 pairs give $U + U$, $U \setminus \text{times} U$, $U \setminus \text{to} U \setminus \text{triangleleft} U$; V is a second universal domain.

Lean 4 source

```
theorem V_isUniversal : IsUniversal.{u} V
```

Exercise 8.18

All six halves of Proposition 8.10, collected as one statement.

Lean 4 source

```
theorem exercise_8_18
```

Exercise 8.19

From $T \setminus \text{triangleleft} E$ and $E \setminus \text{to} E \setminus \text{triangleleft} E$ one gets $E + E \setminus \text{triangleleft} E$ and $E \setminus \text{times} E \setminus \text{triangleleft} E$.

Lean 4 source

```
theorem exercise_8_19 {a : Type u} {E : NeighborhoodSystem a}
```

Exercise 8.20

Every domain satisfies $D \setminus \text{triangleleft} D \setminus \text{times} D$ and $D \setminus \text{triangleleft} (D \setminus \text{to} D)$.

Lean 4 source

```
theorem exercise_8_20
```

Exercise 8.21

A computable operator $\lambda a. a^\S$ on projections of U with $D_{a^\S} \cong (D_a)^\S$.

Lean 4 source

```
noncomputable def aSharp (a : ApproximableMap U U) : ApproximableMap U U
```

Exercise 8.21(a)

Construction, computability, and $a \setminus le I \setminus implies a^\S \leq I$.

Lean 4 source

```
theorem aSharp_isComputable {a : ApproximableMap U U}
```

Exercise 8.21(b)

Idempotence $a = a \setminus circa \setminus implies a^\S = a^\S \circ a^\S$, completing projection-closure.

Lean 4 source

```
theorem isProjection_aSharp {a : ApproximableMap U U} (ha : IsProjection a) :
```

Exercise 8.21(c)

The tree functor on maps identifies $D_{a^\S}$ with $(D_a)^\S$.

Lean 4 source

```
theorem dsharpMap_self_eq_aSharpInner (a : ApproximableMap U U) :
```

Exercise 8.22

Both $B \setminus triangleleft eq C$ and $C \setminus triangleleft eq B$ hold for Example 6.2's $B \setminus cong B + B$ and $C \setminus cong \mathbf{1} + C + C$.

Lean 4 source

```
theorem B_triangleleft eq_C : B ? C
```

Exercise 8.23

If t sends finitary projections to finitary projections with $D_{t(a)} \cong T(D_a)$, then $\text{fix}(t)$ solves $D \setminus cong T(D)$.

Lean 4 source

```
theorem isFinitaryProjection_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E))
```

Exercise 8.24

Two binary constructs yield a pair of effectively presented domains, via 8.23 on the product function space.

Lean 4 source

```
theorem isFinitaryProjection_aStar_bStar
```

Exercise 8.25

A non-trivial $D \setminus cong D \setminus to D$, via $D \setminus cong D \setminus to U^\infty$ and $U^\infty \times U^\infty \cong U^\infty$.

Lean 4 source

```
theorem exercise_8_25 :
```

Exercise 8.26

Untyped λ -calculus hosts in U via $i_\rightarrow, j_\rightarrow$; typed terms retranslate by sandwiching with projections.

Lean 4 source

```
theorem Uapply_Ulam (f : ApproximableMap U U) (x : U.Element) :
```

Exercise 8.27

$\Pi(d)$ is a finitary projection for every $d : U \setminus to U$, not only polymorphic d .

Lean 4 source

```
theorem isFinitaryProjection_piU (d : ApproximableMap U U) : IsFinitaryProjection (piU d)
```

Exercise 8.27(a)

sub as a combinator on U , and $\Pi(d)$ is a projection for every d .

Lean 4 source

```
theorem isProjection_piU (d : ApproximableMap U U) : IsProjection (piU d)
```

Exercise 8.27(b)

$\Pi(d)$ is finitary, via Theorem 8.5's step-closure formula rather than a new dependent-product domain.

Lean 4 source

```
theorem isFinitaryProjection_piU (d : ApproximableMap U U) : IsFinitaryProjection (piU d)
```

Exercise 8.27(b)(0)

Reduces finitariness of $\text{piU } d$ to that of $\text{piD } d$.

Lean 4 source

```
noncomputable def piUFixIso
```

Exercise 8.27(b)(1)

Finitariness of $\text{piD } d$ follows from Scott's formula (ii).

Lean 4 source

```
theorem isFinitaryProjection_piD_of_formula
```

Exercise 8.27(b)(2)

Unwinds $X (\text{piD } d) Y$ through curry and eval.

Lean 4 source

```
noncomputable def piD
```

Exercise 8.27(b)(3)

Proves formula (ii) for $\text{piD } d$.

Lean 4 source

```
theorem hii_piD
```

Exercise 8.27(b)(3)(a)

The easy half: membership reduces to piDApply .

Lean 4 source

```
theorem piD_toElementMap_mem_iff
```

Exercise 8.27(b)(3)(b)

The hard half: compactness descent produces a self-related $X \setminus \text{subsetq } Y$.

Lean 4 source

theorem hii_hard_direction

Exercise 8.27(b)(4)

Assembles formula (ii) into IsFinitaryProjection (piU d).

[Lean 4 source](#)

theorem isFinitaryProjection_piD

Exercise 8.27(b)(5)

The finitary half holds for every d , not only polymorphic d .

[Lean 4 source](#)

theorem isFinitaryProjection_piU

4 Acknowledgments

- **Dana Scott** — *Lectures on a Mathematical Theory of Computation* (PRG-19) [Sco81], the monograph this development formalizes in full, including every exercise.

4.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [COPE24]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`):

- **Cursor [Cur26]** — agent-assisted editing in the Cursor IDE across the whole of this development: formalizing Scott’s 1980 neighborhood-system layer in Lean 4 / mathlib, lake build repair, transcription and upkeep of `sources/PRG19.md`, drafting and maintaining this narrative (`arxiv.md`), and tracking the per-exercise formalized inventory. Generated Lean was provisional until it compiled sorry-free under the pinned toolchain.
- **Cursor Composer [Cmp26]** — unattended multi-hour autorun sessions on Exercise 7.22 (regular-event/automata combinators over the neighborhood system `Ssys`): scaffolding, dependency-ordered module wiring, and routine proof obligations where the strategy was already fixed, under a checklist-driven playbook (`Exercise722-Composer-Playbook.md`) that escalated the hardest sub-goals to a higher-reasoning model rather than looping on Composer alone.
- **Anthropic Claude (Sonnet family) [Son26]** — the primary day-to-day formalization and proof-engineering model throughout this project, used in Cursor at varying reasoning tiers: inventory and narrative maintenance, module wiring, choice-discipline audits (`#print axioms`), and the majority of the Lean proof obligations across Lectures I-VIII, including this file’s own housekeeping (Exercises 8.25-8.27, the `arxiv.md` “Lean Code” appendix repair, and this LaTeX/PDF pipeline).
- **Anthropic Claude Opus [Ant26]** — selective escalation for the heaviest proof obligations flagged in the Composer playbook as needing higher reasoning depth than routine autorun work (e.g. Exercise 7.22’s full language-equivalence decider). Every emitted proof term was checked by the Lean kernel regardless of which model produced it.

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

4.2 Artifact availability

The development [ER80] is at github.com/catskillsresearch/scott1980. The Palomar-preserved copy linked in the Appendix is <https://github.com/PalomarArchive/catskillsresearch-scott1980-01865c711747/tree/ef6c05671582b7883ecd40b7081b642ce5351963>. Rebuild the formalization and this document with:

```
lake exe cache get
lake build Scott1980
python3 scripts/generate_lecture_mermaid.py --write # sync chapter + per-lecture mermaid
          blocks in arxiv.md
bash scripts/build_arxiv_pdf.sh # narrative + Palomar-link appendix -> arxiv.pdf
bash scripts/build_arxiv_pdf.sh --pdf-only # PDF only when arxiv.tex already current
```

5 List of Figures

Nine mermaid diagrams: the chapter-import hierarchy, then one Lean-module dependency chart per lecture.

List of Figures

1	Chapter inclusion hierarchy (Lean imports).	4
2	Lean module dependencies for Lecture I: Domains by Neighbourhoods.	7
3	Lean module dependencies for Lecture II: Approximable Mappings.	13
4	Lean module dependencies for Lecture III: Domain Constructs.	17
5	Lean module dependencies for Lecture IV: Fixed Points and Recursion.	23
6	Lean module dependencies for Lecture V: Typed λ -Calculus.	28
7	Lean module dependencies for Lecture VI: Domain Equations.	31
8	Lean module dependencies for Lecture VII: Computability in Effectively Given Domains.	36
9	Lean module dependencies for Lecture VIII: Retracts of the Universal Domain. .	43

6 References

- [Sco69] D. S. Scott. *Lattice-theoretic models for the λ -calculus* (Unpublished manuscript). University of Oxford, 1969.
- [Sco72] D. S. Scott. Continuous lattices. In F. W. Lawvere (Ed.), *Toposes, Algebraic Geometry and Logic* (Lecture Notes in Mathematics, Vol. 274, pp. 97–136). Springer, Berlin, Heidelberg, 1972.
- [Sco81] D. S. Scott. *Lectures on a mathematical theory of computation* (Technical Monograph PRG-19). Oxford University Computing Laboratory, Programming Research Group, May 1981. Lectures delivered Michaelmas Term 1980.
- [Win93] G. Winskel. *The Formal Semantics of Programming Languages*. MIT Press, 1993.

- [ER80] Lars Warren Ericson. *scott1980* (this work). <https://github.com/catskillsresearch/scott1980>.
- [COPE24] Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement*. 2024. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- [Cur26] Anysphere, Inc. *Cursor: AI-native code editor and agent environment*. <https://cursor.com> (accessed 2026).
- [Cmp26] Anysphere, Inc. *Composer*. Model announcement and documentation, <https://cursor.com/docs/models> (accessed 2026).
- [Son26] Anthropic. *Claude Sonnet*. System cards and model documentation, <https://www.anthropic.com/claude>; as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).
- [Ant26] Anthropic. *Claude Opus*. System cards and model documentation, <https://www.anthropic.com/claude>; as integrated in Cursor, <https://cursor.com/docs/models> (accessed 2026).

A Lean sources in the Palomar archive

The complete Lean sources are not reproduced in this note. Each library file below is the copy preserved by Palomar at commit `ef6c05671582b7883ecd40b7081b642ce5351963`. The archive tree is <https://github.com/PalomarArchive/catskillsresearch-scott1980-01865c711747/tree/ef6c05671582b7883>. The working repository is <https://github.com/catskillsresearch/scott1980>.

A.1 `Scott1980.lean`

Root import module. It loads the `neighbourhood-system` library in `Scott1980.lean` import order so that `import Scott1980` typechecks the complete sorry-free development.

A.2 `Scott1980/Neighborhood/Basic.lean`

Lecture I core: `NeighborhoodSystem`, filters `Element`, principal filters, the approximation order, and the nested-or-disjoint constructor.

A.3 `Scott1980/Neighborhood/Example12.lean`

Example 1.2 of PRG-19, as formalized in this module.

A.4 `Scott1980/Neighborhood/Example13.lean`

Example 1.3 of PRG-19, as formalized in this module.

A.5 `Scott1980/Neighborhood/Example14.lean`

Example 1.4 of PRG-19, as formalized in this module.

A.6 `Scott1980/Neighborhood/Example15.lean`

Example 1.5 of PRG-19, as formalized in this module.

A.7 [Scott1980/Neighborhood/ExampleB.lean](#)

Support module `ExampleB` of the Scott 1981 formalization.

A.8 [Scott1980/Neighborhood/Theorem110.lean](#)

Theorem 1.10 of PRG-19, as formalized in this module.

A.9 [Scott1980/Neighborhood/Theorem111.lean](#)

Theorem 1.11 of PRG-19, as formalized in this module.

A.10 [Scott1980/Neighborhood/Exercise112.lean](#)

Exercise 1.12 of PRG-19, as formalized in this module.

A.11 [Scott1980/Neighborhood/Exercise113.lean](#)

Exercise 1.13 of PRG-19, as formalized in this module.

A.12 [Scott1980/Neighborhood/Exercise114.lean](#)

Exercise 1.14 of PRG-19, as formalized in this module.

A.13 [Scott1980/Neighborhood/Exercise115.lean](#)

Exercise 1.15 of PRG-19, as formalized in this module.

A.14 [Scott1980/Neighborhood/Exercise116.lean](#)

Exercise 1.16 of PRG-19, as formalized in this module.

A.15 [Scott1980/Neighborhood/Exercise117.lean](#)

Exercise 1.17 of PRG-19, as formalized in this module.

A.16 [Scott1980/Neighborhood/Exercise118.lean](#)

Exercise 1.18 of PRG-19, as formalized in this module.

A.17 [Scott1980/Neighborhood/Exercise119.lean](#)

Exercise 1.19 of PRG-19, as formalized in this module.

A.18 [Scott1980/Neighborhood/Exercise120.lean](#)

Exercise 1.20 of PRG-19, as formalized in this module.

A.19 [Scott1980/Neighborhood/Exercise121.lean](#)

Exercise 1.21 of PRG-19, as formalized in this module.

A.20 [Scott1980/Neighborhood/Exercise122.lean](#)

Exercise 1.22 of PRG-19, as formalized in this module.

A.21 [Scott1980/Neighborhood/Exercise123.lean](#)

Exercise 1.23 of PRG-19, as formalized in this module.

A.22 [Scott1980/Neighborhood/Exercise124.lean](#)

Exercise 1.24 of PRG-19, as formalized in this module.

A.23 [Scott1980/Neighborhood/Exercise125.lean](#)

Exercise 1.25 of PRG-19, as formalized in this module.

A.24 [Scott1980/Neighborhood/Exercise126.lean](#)

Exercise 1.26 of PRG-19, as formalized in this module.

A.25 [Scott1980/Neighborhood/Exercise127.lean](#)

Exercise 1.27 of PRG-19, as formalized in this module.

A.26 [Scott1980/Neighborhood/Approximable.lean](#)

Lecture II: approximable maps as relations, `toElementMap`, identity, composition, and maps induced by order isomorphisms.

A.27 [Scott1980/Neighborhood/ApproximableExercises.lean](#)

Lecture II exercises on approximable maps: monotonicity, directed suprema, and the `ofMono / ofContinuous` constructors.

A.28 [Scott1980/Neighborhood/Example23.lean](#)

Example 2.3 of PRG-19, as formalized in this module.

A.29 [Scott1980/Neighborhood/Example24.lean](#)

Example 2.4 of PRG-19, as formalized in this module.

A.30 [Scott1980/Neighborhood/Exercise213.lean](#)

Exercise 2.13 of PRG-19, as formalized in this module.

A.31 [Scott1980/Neighborhood/Exercise214.lean](#)

Exercise 2.14 of PRG-19, as formalized in this module.

A.32 [Scott1980/Neighborhood/Exercise215.lean](#)

Exercise 2.15 of PRG-19, as formalized in this module.

A.33 [Scott1980/Neighborhood/Exercise216.lean](#)

Exercise 2.16 of PRG-19, as formalized in this module.

A.34 [Scott1980/Neighborhood/Exercise218.lean](#)

Exercise 2.18 of PRG-19, as formalized in this module.

A.35 [Scott1980/Neighborhood/Exercise220.lean](#)

Exercise 2.20 of PRG-19, as formalized in this module.

A.36 [Scott1980/Neighborhood/Exercise221.lean](#)

Exercise 2.21 of PRG-19, as formalized in this module.

A.37 [Scott1980/Neighborhood/Exercise222.lean](#)

Exercise 2.22 of PRG-19, as formalized in this module.

A.38 [Scott1980/Neighborhood/Product.lean](#)

Lecture III product of neighbourhood systems, pairing, and the projections of Proposition 3.2.

A.39 [Scott1980/Neighborhood/FunctionSpace.lean](#)

Lecture III function space of approximable maps, `eval/curry`, and the cartesian-closed structure.

A.40 [Scott1980/Neighborhood/Exercise314.lean](#)

Exercise 3.14 of PRG-19, as formalized in this module.

A.41 [Scott1980/Neighborhood/Exercise315.lean](#)

Exercise 3.15 of PRG-19, as formalized in this module.

A.42 [Scott1980/Neighborhood/Exercise318.lean](#)

Exercise 3.18 of PRG-19, as formalized in this module.

A.43 [Scott1980/Neighborhood/Exercise319.lean](#)

Exercise 3.19 of PRG-19, as formalized in this module.

A.44 [Scott1980/Neighborhood/Exercise319Sum.lean](#)

Exercise 3.19Sum of PRG-19, as formalized in this module.

A.45 [Scott1980/Neighborhood/Exercise321.lean](#)

Exercise 3.21 of PRG-19, as formalized in this module.

A.46 [Scott1980/Neighborhood/Exercise322.lean](#)

Exercise 3.22 of PRG-19, as formalized in this module.

A.47 [Scott1980/Neighborhood/Exercise323.lean](#)

Exercise 3.23 of PRG-19, as formalized in this module.

A.48 [Scott1980/Neighborhood/Exercise324.lean](#)

Exercise 3.24 of PRG-19, as formalized in this module.

A.49 [Scott1980/Neighborhood/Exercise316.lean](#)

Exercise 3.16 of PRG-19, as formalized in this module.

A.50 [Scott1980/Neighborhood/Exercise317.lean](#)

Exercise 3.17 of PRG-19, as formalized in this module.

A.51 [Scott1980/Neighborhood/Exercise324Iter.lean](#)

Exercise 3.24Iter of PRG-19, as formalized in this module.

A.52 [Scott1980/Neighborhood/Exercise324Distrib.lean](#)

Exercise 3.24Distrib of PRG-19, as formalized in this module.

A.53 [Scott1980/Neighborhood/Exercise325.lean](#)

Exercise 3.25 of PRG-19, as formalized in this module.

A.54 [Scott1980/Neighborhood/Exercise327.lean](#)

Exercise 3.27 of PRG-19, as formalized in this module.

A.55 [Scott1980/Neighborhood/Exercise326.lean](#)

Exercise 3.26 of PRG-19, as formalized in this module.

A.56 [Scott1980/Neighborhood/Exercise326Sum.lean](#)

Exercise 3.26Sum of PRG-19, as formalized in this module.

A.57 [Scott1980/Neighborhood/Exercise328.lean](#)

Exercise 3.28 of PRG-19, as formalized in this module.

A.58 [Scott1980/Neighborhood/Theorem41.lean](#)

Theorem 4.1 of PRG-19, as formalized in this module.

A.59 [Scott1980/Neighborhood/Example43.lean](#)

Example 4.3 of PRG-19, as formalized in this module.

A.60 [Scott1980/Neighborhood/Example44.lean](#)

Example 4.4 of PRG-19, as formalized in this module.

A.61 [Scott1980/Neighborhood/Theorem46.lean](#)

Theorem 4.6 of PRG-19, as formalized in this module.

A.62 [Scott1980/Neighborhood/Exercise407.lean](#)

Exercise 4.07 of PRG-19, as formalized in this module.

A.63 [Scott1980/Neighborhood/Exercise408.lean](#)

Exercise 4.08 of PRG-19, as formalized in this module.

A.64 [Scott1980/Neighborhood/Exercise409.lean](#)

Exercise 4.09 of PRG-19, as formalized in this module.

A.65 [Scott1980/Neighborhood/Exercise410.lean](#)

Exercise 4.10 of PRG-19, as formalized in this module.

A.66 [Scott1980/Neighborhood/Exercise411.lean](#)

Exercise 4.11 of PRG-19, as formalized in this module.

A.67 [Scott1980/Neighborhood/Exercise412.lean](#)

Exercise 4.12 of PRG-19, as formalized in this module.

A.68 [Scott1980/Neighborhood/Exercise413.lean](#)

Exercise 4.13 of PRG-19, as formalized in this module.

A.69 [Scott1980/Neighborhood/Exercise414.lean](#)

Exercise 4.14 of PRG-19, as formalized in this module.

A.70 [Scott1980/Neighborhood/Exercise415.lean](#)

Exercise 4.15 of PRG-19, as formalized in this module.

A.71 [Scott1980/Neighborhood/Exercise416.lean](#)

Exercise 4.16 of PRG-19, as formalized in this module.

A.72 [Scott1980/Neighborhood/Exercise417.lean](#)

Exercise 4.17 of PRG-19, as formalized in this module.

A.73 [Scott1980/Neighborhood/Exercise418.lean](#)

Exercise 4.18 of PRG-19, as formalized in this module.

A.74 [Scott1980/Neighborhood/Exercise419.lean](#)

Exercise 4.19 of PRG-19, as formalized in this module.

A.75 [Scott1980/Neighborhood/Exercise420.lean](#)

Exercise 4.20 of PRG-19, as formalized in this module.

A.76 [Scott1980/Neighborhood/Exercise421.lean](#)

Exercise 4.21 of PRG-19, as formalized in this module.

A.77 [Scott1980/Neighborhood/Exercise422.lean](#)

Exercise 4.22 of PRG-19, as formalized in this module.

A.78 [Scott1980/Neighborhood/Exercise423.lean](#)

Exercise 4.23 of PRG-19, as formalized in this module.

A.79 [Scott1980/Neighborhood/Exercise424.lean](#)

Exercise 4.24 of PRG-19, as formalized in this module.

A.80 [Scott1980/Neighborhood/Exercise425.lean](#)

Exercise 4.25 of PRG-19, as formalized in this module.

A.81 [Scott1980/Neighborhood/Table55.lean](#)

Lecture V Table 5.5: combinators written in λ -notation.

A.82 [Scott1980/Neighborhood/Theorem51.lean](#)

Theorem 5.1 of PRG-19, as formalized in this module.

A.83 [Scott1980/Neighborhood/Theorem52.lean](#)

Theorem 5.2 of PRG-19, as formalized in this module.

A.84 [Scott1980/Neighborhood/Proposition53.lean](#)

Proposition 5.3 of PRG-19, as formalized in this module.

A.85 [Scott1980/Neighborhood/Proposition54.lean](#)

Proposition 5.4 of PRG-19, as formalized in this module.

A.86 [Scott1980/Neighborhood/Theorem56.lean](#)

Theorem 5.6 of PRG-19, as formalized in this module.

A.87 [Scott1980/Neighborhood/Theorem56Full.lean](#)

Theorem 5.6Full of PRG-19, as formalized in this module.

A.88 [Scott1980/Neighborhood/Exercise507.lean](#)

Exercise 5.07 of PRG-19, as formalized in this module.

A.89 [Scott1980/Neighborhood/Exercise508.lean](#)

Exercise 5.08 of PRG-19, as formalized in this module.

A.90 [Scott1980/Neighborhood/Exercise509.lean](#)

Exercise 5.09 of PRG-19, as formalized in this module.

A.91 [Scott1980/Neighborhood/Exercise510.lean](#)

Exercise 5.10 of PRG-19, as formalized in this module.

A.92 [Scott1980/Neighborhood/Exercise511.lean](#)

Exercise 5.11 of PRG-19, as formalized in this module.

A.93 [Scott1980/Neighborhood/Exercise512.lean](#)

Exercise 5.12 of PRG-19, as formalized in this module.

A.94 [Scott1980/Neighborhood/Exercise513.lean](#)

Exercise 5.13 of PRG-19, as formalized in this module.

A.95 [Scott1980/Neighborhood/Exercise514.lean](#)

Exercise 5.14 of PRG-19, as formalized in this module.

A.96 [Scott1980/Neighborhood/Exercise515.lean](#)

Exercise 5.15 of PRG-19, as formalized in this module.

A.97 [Scott1980/Neighborhood/Exercise516.lean](#)

Exercise 5.16 of PRG-19, as formalized in this module.

A.98 [Scott1980/Neighborhood/Exercise516ThueMorse.lean](#)

Exercise 5.16ThueMorse of PRG-19, as formalized in this module.

A.99 [Scott1980/Neighborhood/Exercise516Overlap.lean](#)

Exercise 5.16Overlap of PRG-19, as formalized in this module.

A.100 [Scott1980/Neighborhood/Example61.lean](#)

Example 6.1 of PRG-19, as formalized in this module.

A.101 [Scott1980/Neighborhood/Example62.lean](#)

Example 6.2 of PRG-19, as formalized in this module.

A.102 [Scott1980/Neighborhood/Example62C.lean](#)

Example 6.2C of PRG-19, as formalized in this module.

A.103 [Scott1980/Neighborhood/Example62A.lean](#)

Example 6.2A of PRG-19, as formalized in this module.

A.104 [Scott1980/Neighborhood/Example62Regular.lean](#)

Example 6.2Regular of PRG-19, as formalized in this module.

A.105 [Scott1980/Neighborhood/Definition63.lean](#)

Definition 6.3 of PRG-19, as formalized in this module.

A.106 [Scott1980/Neighborhood/Proposition66.lean](#)

Proposition 6.6 of PRG-19, as formalized in this module.

A.107 [Scott1980/Neighborhood/Proposition67.lean](#)

Proposition 6.7 of PRG-19, as formalized in this module.

A.108 [Scott1980/Neighborhood/Definition68.lean](#)

Definition 6.8 of PRG-19, as formalized in this module.

A.109 [Scott1980/Neighborhood/Theorem69.lean](#)

Theorem 6.9 of PRG-19, as formalized in this module.

A.110 [Scott1980/Neighborhood/Definition610.lean](#)

Definition 6.10 of PRG-19, as formalized in this module.

A.111 [Scott1980/Neighborhood/Proposition611.lean](#)

Proposition 6.11 of PRG-19, as formalized in this module.

A.112 [Scott1980/Neighborhood/Proposition612.lean](#)

Proposition 6.12 of PRG-19, as formalized in this module.

A.113 [Scott1980/Neighborhood/Definition613.lean](#)

Definition 6.13 of PRG-19, as formalized in this module.

A.114 [Scott1980/Neighborhood/Theorem614.lean](#)

Theorem 6.14 of PRG-19, as formalized in this module.

A.115 [Scott1980/Neighborhood/Lemma615.lean](#)

Lemma 6.15 of PRG-19, as formalized in this module.

A.116 [Scott1980/Neighborhood/Theorem616.lean](#)

Theorem 6.16 of PRG-19, as formalized in this module.

A.117 [Scott1980/Neighborhood/Exercise617.lean](#)

Exercise 6.17 of PRG-19, as formalized in this module.

A.118 [Scott1980/Neighborhood/Exercise617Gen.lean](#)

Exercise 6.17Gen of PRG-19, as formalized in this module.

A.119 [Scott1980/Neighborhood/Exercise618.lean](#)

Exercise 6.18 of PRG-19, as formalized in this module.

A.120 [Scott1980/Neighborhood/Exercise619.lean](#)

Exercise 6.19 of PRG-19, as formalized in this module.

A.121 [Scott1980/Neighborhood/Exercise619PartB.lean](#)

Exercise 6.19PartB of PRG-19, as formalized in this module.

A.122 [Scott1980/Neighborhood/Exercise621.lean](#)

Exercise 6.21 of PRG-19, as formalized in this module.

A.123 [Scott1980/Neighborhood/Exercise622.lean](#)

Exercise 6.22 of PRG-19, as formalized in this module.

A.124 [Scott1980/Neighborhood/Exercise623.lean](#)

Exercise 6.23 of PRG-19, as formalized in this module.

A.125 [Scott1980/Neighborhood/Exercise624.lean](#)

Exercise 6.24 of PRG-19, as formalized in this module.

A.126 [Scott1980/Neighborhood/Exercise625.lean](#)

Exercise 6.25 of PRG-19, as formalized in this module.

A.127 [Scott1980/Neighborhood/Exercise626.lean](#)

Exercise 6.26 of PRG-19, as formalized in this module.

A.128 [Scott1980/Neighborhood/Exercise627.lean](#)

Exercise 6.27 of PRG-19, as formalized in this module.

A.129 [Scott1980/Neighborhood/Exercise628.lean](#)

Exercise 6.28 of PRG-19, as formalized in this module.

A.130 [Scott1980/Neighborhood/Exercise629.lean](#)

Exercise 6.29 of PRG-19, as formalized in this module.

A.131 [Scott1980/Neighborhood/Recursive.lean](#)

Choice-free recursion theory used from Lecture VII: `Nat.Primrec` pairing, `RecDecidable`/`REPred`, and their closure lemmas.

A.132 [Scott1980/Neighborhood/Definition71.lean](#)

Definition 7.1 of PRG-19, as formalized in this module.

A.133 [Scott1980/Neighborhood/Definition72.lean](#)

Definition 7.2 of PRG-19, as formalized in this module.

A.134 [Scott1980/Neighborhood/Theorem74.lean](#)

Theorem 7.4 of PRG-19, as formalized in this module.

A.135 [Scott1980/Neighborhood/Theorem75.lean](#)

Theorem 7.5 of PRG-19, as formalized in this module.

A.136 [Scott1980/Neighborhood/Theorem76.lean](#)

Theorem 7.6 of PRG-19, as formalized in this module.

A.137 [Scott1980/Neighborhood/Proposition77.lean](#)

Proposition 7.7 of PRG-19, as formalized in this module.

A.138 [Scott1980/Neighborhood/Combinators77.lean](#)

Proposition 7.7 combinators: the computable maps used to close effectively given domains under the Lecture III constructs.

A.139 [Scott1980/Neighborhood/Example78.lean](#)

Example 7.8 of PRG-19, as formalized in this module.

A.140 [Scott1980/Neighborhood/Definition79.lean](#)

Definition 7.9 of PRG-19, as formalized in this module.

A.141 [Scott1980/Neighborhood/Proposition710.lean](#)

Proposition 7.10 of PRG-19, as formalized in this module.

A.142 [Scott1980/Neighborhood/Definition711.lean](#)

Definition 7.11 of PRG-19, as formalized in this module.

A.143 [Scott1980/Neighborhood/Proposition712.lean](#)

Proposition 7.12 of PRG-19, as formalized in this module.

A.144 [Scott1980/Neighborhood/Exercise713.lean](#)

Exercise 7.13 of PRG-19, as formalized in this module.

A.145 [Scott1980/Neighborhood/Exercise714.lean](#)

Exercise 7.14 of PRG-19, as formalized in this module.

A.146 [Scott1980/Neighborhood/Exercise715.lean](#)

Exercise 7.15 of PRG-19, as formalized in this module.

A.147 [Scott1980/Neighborhood/Exercise716.lean](#)

Exercise 7.16 of PRG-19, as formalized in this module.

A.148 [Scott1980/Neighborhood/Exercise717.lean](#)

Exercise 7.17 of PRG-19, as formalized in this module.

A.149 [Scott1980/Neighborhood/Exercise717Part2.lean](#)

Support module `Exercise717Part2` of the Scott 1981 formalization.

A.150 [Scott1980/Neighborhood/Exercise718.lean](#)

Exercise 7.18 of PRG-19, as formalized in this module.

A.151 [Scott1980/Neighborhood/Exercise719.lean](#)

Exercise 7.19 of PRG-19, as formalized in this module.

A.152 [Scott1980/Neighborhood/Exercise720.lean](#)

Exercise 7.20 of PRG-19, as formalized in this module.

A.153 [Scott1980/Neighborhood/Exercise721.lean](#)

Exercise 7.21 of PRG-19, as formalized in this module.

A.154 [Scott1980/Neighborhood/Exercise722.lean](#)

Exercise 7.22 algebraic core: `Ssys`, multiplication, and the embedding into streams.

A.155 [Scott1980/Neighborhood/Exercise722Regular.lean](#)

Exercise 7.22 regular events: `SExpr` syntax, denotation, and the regular-event characterization.

A.156 [Scott1980/Neighborhood/Exercise722DFA.lean](#)

Exercise 7.22 leaf DFAs and boolean combinations.

A.157 [Scott1980/Neighborhood/Exercise722Cat.lean](#)

Exercise 7.22 concatenation ε NFA.

A.158 [Scott1980/Neighborhood/Exercise722Decide.lean](#)

Exercise 7.22 emptiness and consistency deciders.

A.159 [Scott1980/Neighborhood/Exercise722Words.lean](#)

Exercise 7.22 bounded word enumeration.

A.160 [Scott1980/Neighborhood/Exercise722Equiv.lean](#)

Exercise 7.22 language-equivalence deciders (`subsetB / interEqB`).

A.161 [Scott1980/Neighborhood/Exercise722Presentation.lean](#)

Exercise 7.22 computable presentation of `Ssys`.

A.162 [Scott1980/Neighborhood/Exercise723.lean](#)

Exercise 7.23: computable operations on the powerset domain `PN`.

A.163 [Scott1980/Neighborhood/Exercise724.lean](#)

Exercise 7.24: the LUCID domain L , the isomorphism $\|L\| \simeq \backslash Gamma$, and computable combinators.

A.164 [Scott1980/Neighborhood/Definition81.lean](#)

Definition 8.1 of PRG-19, as formalized in this module.

A.165 [Scott1980/Neighborhood/Proposition82.lean](#)

Proposition 8.2 of PRG-19, as formalized in this module.

A.166 [Scott1980/Neighborhood/Definition83.lean](#)

Definition 8.3 of PRG-19, as formalized in this module.

A.167 [Scott1980/Neighborhood/Theorem85.lean](#)

Theorem 8.5 of PRG-19, as formalized in this module.

A.168 [Scott1980/Neighborhood/Theorem86.lean](#)

Theorem 8.6 of PRG-19, as formalized in this module.

A.169 [Scott1980/Neighborhood/Theorem86c.lean](#)

Theorem 8.6c of PRG-19, as formalized in this module.

A.170 [Scott1980/Neighborhood/Example84.lean](#)

Example 8.4 of PRG-19, as formalized in this module.

A.171 [Scott1980/Neighborhood/Example84b.lean](#)

Example 8.4b of PRG-19, as formalized in this module.

A.172 [Scott1980/Neighborhood/Definition87.lean](#)

Definition 8.7 of PRG-19, as formalized in this module.

A.173 [Scott1980/Neighborhood/RationalPrimrec.lean](#)

A choice-free Gödel numbering of $\mathbb{Q}$ with primitive-recursive comparison.

A.174 [Scott1980/Neighborhood/RecursiveCross.lean](#)

A choice-free primitive-recursive combination of two coded lists.

A.175 [Scott1980/Neighborhood/IntervalPrimrec.lean](#)

Theorem 8.8(b): primitive-recursive interval-code arithmetic for $\mathcal{U}$.

A.176 [Scott1980/Neighborhood/UComputablePresentation.lean](#)

Theorem 8.8(b): $\mathcal{U}$ is effectively given.

A.177 [Scott1980/Neighborhood/SplitU.lean](#)

Theorem 8.8(b): an explicit deterministic bisection of $\mathcal{U}$.

A.178 [Scott1980/Neighborhood/DAtomDecidable.lean](#)

Theorem 8.8(b): decidable emptiness of D -side atoms.

A.179 [Scott1980/Neighborhood/Theorem88.lean](#)

Theorem 8.8 of PRG-19, as formalized in this module.

A.180 [Scott1980/Neighborhood/Theorem88a.lean](#)

Theorem 8.8a of PRG-19, as formalized in this module.

A.181 [Scott1980/Neighborhood/Theorem88b.lean](#)

Theorem 8.8b of PRG-19, as formalized in this module.

A.182 [Scott1980/Neighborhood/Theorem88c.lean](#)

Theorem 8.8c of PRG-19, as formalized in this module.

A.183 [Scott1980/Neighborhood/Theorem88d.lean](#)

Theorem 8.8d of PRG-19, as formalized in this module.

A.184 [Scott1980/Neighborhood/Theorem88e.lean](#)

Theorem 8.8e of PRG-19, as formalized in this module.

A.185 [Scott1980/Neighborhood/Theorem88f.lean](#)

Theorem 8.8f of PRG-19, as formalized in this module.

A.186 [Scott1980/Neighborhood/Theorem88g.lean](#)

Theorem 8.8g of PRG-19, as formalized in this module.

A.187 [Scott1980/Neighborhood/Theorem88h.lean](#)

Theorem 8.8h of PRG-19, as formalized in this module.

A.188 [Scott1980/Neighborhood/Theorem88i.lean](#)

Theorem 8.8i of PRG-19, as formalized in this module.

A.189 [Scott1980/Neighborhood/Theorem88j.lean](#)

Theorem 8.8j of PRG-19, as formalized in this module.

A.190 [Scott1980/Neighborhood/Theorem88k.lean](#)

Theorem 8.8k of PRG-19, as formalized in this module.

A.191 [Scott1980/Neighborhood/Theorem88l.lean](#)

Theorem 8.8l of PRG-19, as formalized in this module.

A.192 [Scott1980/Neighborhood/Theorem88m.lean](#)

Theorem 8.8m of PRG-19, as formalized in this module.

A.193 [Scott1980/Neighborhood/Theorem88n.lean](#)

Theorem 8.8n of PRG-19, as formalized in this module.

A.194 [Scott1980/Neighborhood/Definition89.lean](#)

Definition 8.9 of PRG-19, as formalized in this module.

A.195 [Scott1980/Neighborhood/Proposition810.lean](#)

Proposition 8.10 of PRG-19, as formalized in this module.

A.196 [Scott1980/Neighborhood/Proposition810b.lean](#)

Proposition 8.10b of PRG-19, as formalized in this module.

A.197 [Scott1980/Neighborhood/Exercise811.lean](#)

Exercise 8.11 of PRG-19, as formalized in this module.

A.198 [Scott1980/Neighborhood/Exercise812.lean](#)

Exercise 8.12 of PRG-19, as formalized in this module.

A.199 [Scott1980/Neighborhood/LevelSetPrimrec.lean](#)

Exercise 8.12(b): a computable presentation of V .

A.200 [Scott1980/Neighborhood/MinLevel.lean](#)

Exercise 8.12: minimal-level canonicalization.

A.201 [Scott1980/Neighborhood/Exercise812c.lean](#)

Exercise 8.12c of PRG-19, as formalized in this module.

A.202 [Scott1980/Neighborhood/Exercise812cYseq.lean](#)

Exercise 8.12cYseq of PRG-19, as formalized in this module.

A.203 [Scott1980/Neighborhood/Exercise812d.lean](#)

Exercise 8.12d of PRG-19, as formalized in this module.

A.204 [Scott1980/Neighborhood/Exercise812e.lean](#)

Exercise 8.12e of PRG-19, as formalized in this module.

A.205 [Scott1980/Neighborhood/Exercise812eD.lean](#)

Exercise 8.12eD of PRG-19, as formalized in this module.

A.206 [Scott1980/Neighborhood/SplitV.lean](#)

Exercise 8.12: a computable canonical bisection of V .

A.207 [Scott1980/Neighborhood/VDiff.lean](#)

Exercise 8.12: a computable V -side set difference.

A.208 [Scott1980/Neighborhood/UBisection2.lean](#)

Exercise 8.12(f): a presentation-independent canonical bisection of $\mathcal{U}$.

A.209 [Scott1980/Neighborhood/Exercise812f.lean](#)

Exercise 8.12f of PRG-19, as formalized in this module.

A.210 [Scott1980/Neighborhood/Exercise812g1a.lean](#)

Support module Exercise812g1a of the Scott 1981 formalization.

A.211 [Scott1980/Neighborhood/Exercise812g1b.lean](#)

Support module Exercise812g1b of the Scott 1981 formalization.

A.212 [Scott1980/Neighborhood/Exercise812g2.lean](#)

Support module Exercise812g2 of the Scott 1981 formalization.

A.213 [Scott1980/Neighborhood/Exercise812g4.lean](#)

Support module Exercise812g4 of the Scott 1981 formalization.

A.214 [Scott1980/Neighborhood/Exercise813a.lean](#)

Exercise 8.13a of PRG-19, as formalized in this module.

A.215 [Scott1980/Neighborhood/Exercise813b.lean](#)

Exercise 8.13b of PRG-19, as formalized in this module.

A.216 [Scott1980/Neighborhood/Exercise813c.lean](#)

Exercise 8.13c of PRG-19, as formalized in this module.

A.217 [Scott1980/Neighborhood/Exercise814.lean](#)

Exercise 8.14 of PRG-19, as formalized in this module.

A.218 [Scott1980/Neighborhood/Exercise815.lean](#)

Exercise 8.15 of PRG-19, as formalized in this module.

A.219 [Scott1980/Neighborhood/Exercise816.lean](#)

Exercise 8.16 of PRG-19, as formalized in this module.

A.220 [Scott1980/Neighborhood/Exercise817.lean](#)

Exercise 8.17 of PRG-19, as formalized in this module.

A.221 [Scott1980/Neighborhood/Exercise818.lean](#)

Exercise 8.18 of PRG-19, as formalized in this module.

A.222 [Scott1980/Neighborhood/Exercise819.lean](#)

Exercise 8.19 of PRG-19, as formalized in this module.

A.223 [Scott1980/Neighborhood/Exercise820.lean](#)

Exercise 8.20 of PRG-19, as formalized in this module.

A.224 [Scott1980/Neighborhood/Exercise821.lean](#)

Exercise 8.21 of PRG-19, as formalized in this module.

A.225 [Scott1980/Neighborhood/Exercise822.lean](#)

Exercise 8.22 of PRG-19, as formalized in this module.

A.226 [Scott1980/Neighborhood/Exercise823.lean](#)

Exercise 8.23 of PRG-19, as formalized in this module.

A.227 [Scott1980/Neighborhood/Exercise824.lean](#)

Exercise 8.24 of PRG-19, as formalized in this module.

A.228 [Scott1980/Neighborhood/Exercise825Unit.lean](#)

Exercise 8.25Unit of PRG-19, as formalized in this module.

A.229 [Scott1980/Neighborhood/Exercise825Congr.lean](#)

Exercise 8.25Congr of PRG-19, as formalized in this module.

A.230 [Scott1980/Neighborhood/Exercise825Closing.lean](#)

Exercise 8.25Closing of PRG-19, as formalized in this module.

A.231 [Scott1980/Neighborhood/Exercise825Pow.lean](#)

Exercise 8.25Pow of PRG-19, as formalized in this module.

A.232 [Scott1980/Neighborhood/Exercise825Embed.lean](#)

Exercise 8.25Embed of PRG-19, as formalized in this module.

A.233 [Scott1980/Neighborhood/Exercise825FixedPoint.lean](#)

Exercise 8.25FixedPoint of PRG-19, as formalized in this module.

A.234 [Scott1980/Neighborhood/Exercise825.lean](#)

Exercise 8.25 of PRG-19, as formalized in this module.

A.235 [Scott1980/Neighborhood/Exercise826.lean](#)

Exercise 8.26 of PRG-19, as formalized in this module.

A.236 [Scott1980/Neighborhood/Exercise827.lean](#)

Exercise 8.27 of PRG-19, as formalized in this module.

A.237 [Scott1980/Neighborhood/PalomarExamples.lean](#)

Palomar corollaries `P4_embeds` and `Ssys_embeds`: Theorem 8.8 specialized to Example 1.5 and Exercise 7.22's system S .

A.238 [Challenge.lean](#)

Palomar statement of record. It locks the first sentence of Theorem 8.8 as `theorem_8_8` together with the new formal corollaries `P4_embeds` and `Ssys_embeds`, using only `Mathlib`.

A.239 [Solution.lean](#)

Palomar realization. It imports `Scott1980.Neighborhood.PalomarExamples` so the compared declarations are the sorry-free library proofs.